

**INSTITUTO FEDERAL CATARINENSE - IFC  
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO**

**ANDRÉ VITOR BASTOS DE MACÊDO**

**PATHFINDING EM MALHAS 3D : Análise estatística e comparativa**

**Blumenau  
2026**

ANDRÉ VITOR BASTOS DE MACÊDO

**PATHFINDING EM MALHAS 3D : Análise estatística e comparativa**

Pré-projeto de Trabalho de Conclusão de Curso  
apresentado ao curso de Bacharelado em Ciência  
da Computação do Instituto Federal Catarinense,  
como requisito parcial para a obtenção do grau de  
Bacharel.

**Orientador(a):** Prof. Paulo César Rodacki Gomes

## LISTA DE ILUSTRAÇÕES

|           |                                                                                       |    |
|-----------|---------------------------------------------------------------------------------------|----|
| Figura 1  | Diagrama de classes do padrão <i>Composite</i> .....                                  | 15 |
| Figura 2  | Diagrama de classes do padrão <i>Singleton</i> .....                                  | 15 |
| Figura 3  | Etapas de geração de um labirinto perfeito utilizando o algoritmo de<br>Kruskal ..... | 21 |
| Figura 4  | Exemplo de mapa de altura .....                                                       | 22 |
| Figura 5  | Conversão de um mapa de altura para uma malha 3D .....                                | 23 |
| Figura 6  | Exemplo de malha gerada a partir de um mapa de altura .....                           | 24 |
| Figura 7  | Exemplo de ruído de Perlin e uma de suas oitavas ( <i>octaves</i> ) .....             | 25 |
| Figura 8  | Ruídos de Perlin com diferentes tamanhos de células .....                             | 26 |
| Figura 9  | Cálculo do produto escalar em uma célula do ruído de Perlin .....                     | 27 |
| Figura 10 | Ruídos de Perlin com diferentes frequências e amplitudes .....                        | 27 |
| Figura 11 | Exemplo de malha gerada a partir de um ruído de Perlin .....                          | 29 |
| Figura 12 | Função de Suavização no Ruído de Perlin .....                                         | 30 |
| Figura 13 | Diagrama de Thread Pool e filas multinível .....                                      | 33 |
| Figura 14 | Diagrama de classe do Composite .....                                                 | 39 |
| Figura 15 | Diagrama de classes da arquitetura do TaskMaster .....                                | 44 |

## **LISTA DE TABELAS**

|          |                                                                             |    |
|----------|-----------------------------------------------------------------------------|----|
| Tabela 1 | Matriz de diferenciação entre a proposta e trabalhos relacionados . . . . . | 36 |
| Tabela 2 | Cronograma de execução das atividades . . . . .                             | 49 |

## LISTA DE ABREVIATURAS E SIGLAS

|                |                                                                                          |
|----------------|------------------------------------------------------------------------------------------|
| <b>A*</b>      | <i>A-Star</i>                                                                            |
| <b>ANOVA</b>   | <i>Analysis of Variance</i><br>(Análise de Variância)                                    |
| <b>API</b>     | <i>Application Programming Interface</i><br>(Interface de Programação de Aplicações)     |
| <b>CPU</b>     | <i>Central Processing Unit</i> (Unidade Central de Processamento)                        |
| <b>EBO</b>     | <i>Element Buffer Object</i>                                                             |
| <b>FPS</b>     | <i>Frames Per Second</i><br>(Quadros por Segundo)                                        |
| <b>GCC</b>     | <i>GNU Compiler Collection</i>                                                           |
| <b>GLFW</b>    | <i>Graphics Library Framework</i>                                                        |
| <b>GLM</b>     | <i>OpenGL Mathematics</i>                                                                |
| <b>GLSL</b>    | <i>OpenGL Shading Language</i>                                                           |
| <b>GPU</b>     | <i>Graphics Processing Unit</i><br>(Unidade de Processamento Gráfico)                    |
| <b>IFCG</b>    | Instituto Federal Catarinense/Computação Gráfica                                         |
| <b>I/O</b>     | <i>Input/Output</i><br>(Entrada/Saída)                                                   |
| <b>IJACSA</b>  | <i>International Journal of Advanced Computer Science and Applications</i>               |
| <b>JPS</b>     | <i>Jump Point Search</i>                                                                 |
| <b>MST</b>     | <i>Minimum Spanning Tree</i><br>(Árvore Geradora Mínima)                                 |
| <b>OpenGL</b>  | <i>Open Graphics Library</i>                                                             |
| <b>PNG</b>     | <i>Portable Network Graphics</i>                                                         |
| <b>RAII</b>    | <i>Resource Acquisition Is Initialization</i><br>(Aquisição de Recursos é Inicialização) |
| <b>RAM</b>     | <i>Random Access Memory</i><br>(Memória de Acesso Aleatório)                             |
| <b>RGB</b>     | <i>Red, Green, Blue</i><br>(Vermelho, Verde, Azul)                                       |
| <b>SIGSEGV</b> | <i>Segmentation Violation</i><br>(Violação de Segmentação)                               |
| <b>STL</b>     | <i>Standard Template Library</i><br>(Biblioteca Padrão de Modelos)                       |

|            |                                |
|------------|--------------------------------|
| <b>TCC</b> | Trabalho de Conclusão de Curso |
| <b>VAO</b> | <i>Vertex Array Object</i>     |
| <b>VBO</b> | <i>Vertex Buffer Object</i>    |

## SUMÁRIO

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1 INTRODUÇÃO</b>                                     | <b>8</b>  |
| <b>2 JUSTIFICATIVA</b>                                  | <b>10</b> |
| <b>3 OBJETIVOS</b>                                      | <b>11</b> |
| 3.1 Objetivos Gerais                                    | 11        |
| 3.2 Objetivos Específicos                               | 11        |
| <b>4 FUNDAMENTAÇÃO TEÓRICA</b>                          | <b>12</b> |
| <b>4.1 Pilha Tecnológica</b>                            | <b>12</b> |
| 4.1.1 Linguagem de Programação                          | 12        |
| 4.1.2 Arquitetura de Programas                          | 14        |
| <b>4.2 Teoria dos Grafos</b>                            | <b>16</b> |
| 4.2.1 Definição e Propriedades dos Grafos               | 16        |
| 4.2.2 Grafos Direcionados e Não Direcionados            | 16        |
| 4.2.3 MST ( <i>Minimum Spanning Tree</i> )              | 17        |
| <b>4.3 Algoritmos de Busca Heurística</b>               | <b>17</b> |
| 4.3.1 Dijkstra                                          | 17        |
| 4.3.2 Heurísticas                                       | 17        |
| 4.3.3 A*                                                | 18        |
| 4.3.4 Jump Point Search (JPS)                           | 18        |
| 4.3.5 Theta*                                            | 18        |
| <b>4.4 Computação Gráfica</b>                           | <b>18</b> |
| 4.4.1 OpenGL API                                        | 19        |
| <b>4.5 Geração de Cenários de Teste</b>                 | <b>20</b> |
| 4.5.1 Labirintos                                        | 20        |
| 4.5.2 Mapas de Altura                                   | 21        |
| 4.5.3 Geração de Ruídos                                 | 24        |
| <b>4.6 Concorrência e Paralelismo</b>                   | <b>30</b> |
| 4.6.1 <i>Multithreading</i> com C++20                   | 30        |
| 4.6.2 Tokens de Parada                                  | 31        |
| 4.6.3 Variáveis Condicionais                            | 31        |
| 4.6.4 Monitores                                         | 31        |
| 4.6.5 <i>Task Scheduler</i>                             | 31        |
| 4.6.6 Fila Multinível                                   | 32        |
| 4.6.7 Segurança de Memória e Compartilhamento de Estado | 33        |
| <b>4.7 Análise Estatística</b>                          | <b>34</b> |
| <b>5 TRABALHOS RELACIONADOS</b>                         | <b>35</b> |
| 5.1 Johansson (2024)                                    | 35        |
| 5.2 Grid Maps Benchmark (2023)                          | 35        |
| 5.3 Matriz de Diferenciação Técnica                     | 36        |
| <b>6 METODOLOGIA</b>                                    | <b>37</b> |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>6.1 Arquitetura do Motor Gráfico (IFCG)</b>                | <b>37</b> |
| 6.1.1 Aplicação de conceitos                                  | 37        |
| 6.1.2 Componentes do Motor Gráfico                            | 38        |
| 6.1.3 Estrutura de Renderização e Gerenciamento de Recursos   | 38        |
| <b>6.2 Implementação da Estrutura de Grafos</b>               | <b>39</b> |
| 6.2.1 Representação de Grafos Direcionados e Não Direcionados | 40        |
| 6.2.2 Algoritmos de Pathfinding                               | 40        |
| 6.2.3 Otimizações e Estruturas de Dados Auxiliares            | 41        |
| <b>6.3 Geração e Processamento dos Cenários</b>               | <b>42</b> |
| 6.3.1 Geração de Malhas 3D Complexas                          | 42        |
| 6.3.2 Processamento de Malhas e Extração de Grafos            | 42        |
| 6.3.3 Configuração de Cenários de Teste e Parâmetros          | 42        |
| <b>6.4 Implementação do Sistema Concorrente</b>               | <b>43</b> |
| 6.4.1 Adaptação para Ambientes de Renderização                | 43        |
| 6.4.2 Implementação de Multithreading e Testes de Estresse    | 43        |
| 6.4.3 Controle de Recursos do Sistema Operacional             | 45        |
| <b>6.5 Desenho Experimental e Coleta de Dados</b>             | <b>45</b> |
| 6.5.1 Ambiente de Teste                                       | 45        |
| 6.5.2 Configuração dos Testes e Métricas Coletadas            | 46        |
| 6.5.3 Execução dos Testes e Registro dos Resultados           | 46        |
| <b>7 RESULTADOS</b>                                           | <b>48</b> |
| <b>8 CRONOGRAMA</b>                                           | <b>49</b> |



## 1 INTRODUÇÃO

A busca de caminhos (ou *pathfinding*) é um dos campos mais tradicionais da Inteligência Artificial aplicada, evoluindo a partir dos algoritmos de busca clássicos em grafos. Como definem Russell; Norvig (2013), os métodos clássicos de busca estruturam problemas em termos de estados, ações e custos, visando encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração sistemática do espaço de busca. Ao longo das últimas décadas, essa fundamentação teórica deixou de ser puramente abstrata e passou a ser aplicada diretamente na computação interativa. Conforme apontam Pardede et al. (2022), os algoritmos de busca tornaram-se componentes cruciais para o desenvolvimento de jogos modernos e simulações em tempo real, nos quais personagens controlados por computador precisam navegar por cenários dinâmicos de forma autônoma e inteligente.

No entanto, a transição das simulações bidimensionais (2D) para os ambientes tridimensionais (3D) gerou um grande salto na complexidade computacional dos algoritmos. Há na literatura científica uma lacuna metodológica significativa, na qual a maioria dos algoritmos de busca é testada e avaliada isoladamente em cenários planos ou ideais, ignorando as restrições físicas de um ambiente real. No espaço 3D real, é preciso considerar a variação de altura, inclinação do relevo e obstáculos tridimensionais complexos. Conforme discutido por Smolka et al. (2019), a navegação em engines 3D reais traz problemas práticos de instabilidade e inconsistências matemáticas nos cálculos de distância. Isso exige o desenvolvimento de funções de suavização (*smoothing*) e modificações nos algoritmos heurísticos tradicionais para garantir que os caminhos gerados sejam tanto realistas quanto viáveis fisicamente.

Este estudo propõe uma análise estatística e comparativa com foco no desempenho de diferentes algoritmos de busca heurística em malhas 3D acidentadas. O objetivo principal é medir métricas de eficiência computacional, como tempo de execução em microssegundos, consumo absoluto de memória RAM e overhead de processamento sob concorrência. Para garantir controle total sobre as variáveis de teste e isolar o impacto do hardware e das chamadas ao sistema operacional, o experimento adota uma infraestrutura de desenvolvimento própria. Tanto o motor de renderização (denominado IFCG) quanto as estruturas de dados de grafos e algoritmos foram implementados do zero em linguagem C++, permitindo extrair métricas de desempenho livres das interferências comuns de engines comerciais.

A otimização de algoritmos de busca em ambientes tridimensionais dinâmicos permanece na fronteira da pesquisa científica na área de computação. Trabalhos recentes, como o de Madushanka; Madushanka (2026), destacam a importância e a necessidade do processamento concorrente (*multithreading*) para calcular caminhos em tempo real sem prejudicar a responsividade visual do motor gráfico. Além disso, a adaptação de técnicas eficientes baseadas em grades uniformes para o espaço volumétrico, como a expansão tridimensional do algoritmo *Jump Point Search* (JPS) estudada por Nobes et al. (2022), demonstra que o desenvolvimento de heurísticas

espaciais avançadas é um tema ativo e de grande interesse para aplicações de grande escala.

## 2 JUSTIFICATIVA

Com este capítulo, pretende-se justificar a relevância e a necessidade do estudo proposto, destacando a importância de uma análise estatística detalhada dos algoritmos de pathfinding em malhas 3D, especialmente considerando as complexidades e desafios dos ambientes reais.

Na área de algoritmos de *pathfinding*, busca de caminhos, a maioria dos estudos se concentra em topologias 2D ou não geométricas e em cenários ideais, onde as condições são controladas e otimizadas para destacar as vantagens de cada algoritmo. No entanto, a transição para ambientes 3D introduz uma série de desafios adicionais, como a complexidade da geometria, a necessidade de lidar com obstáculos tridimensionais e a gestão de recursos computacionais. A falta de análises estatísticas detalhadas sob condições adversas limita a compreensão real do desempenho desses algoritmos em situações práticas, onde otimizações matemáticas e técnicas avançadas podem ter um impacto significativo.

Este estudo se diferencia por adotar uma abordagem aprofundada e detalhada, focando em métricas quantitativas e condições adversas que refletem os desafios do mundo real. Ao invés de uma visão superficial e generalizada, a pesquisa se propõe a investigar o comportamento de algoritmos de busca quando expostos a concorrência computacional e arquiteturas complexas.

Além disso, a construção de uma infraestrutura gráfica e de uma biblioteca de grafos do zero não apenas proporciona um ambiente de teste personalizado, mas também garante um controle total sobre as variáveis e anomalias de hardware que podem afetar os resultados. A correta aplicação teórica e estrutural da base de grafos é fundamental para garantir a validade dos testes empíricos, conforme destacado por Gomes (2022).

Com isso, viu-se a necessidade de um estudo que vá além dos testes em ambientes ideais, oferecendo uma análise estatística detalhada do desempenho dos algoritmos de pathfinding em malhas 3D, contribuindo para a escolha informada de técnicas de navegação em projetos futuros.

### 3 OBJETIVOS

Neste capítulo serão apresentados os objetivos gerais e específicos do estudo, detalhando o que se pretende alcançar com a pesquisa. As metas girarão em torno da implementação de algoritmos de pathfinding, desenvolvimento de uma infraestrutura gráfica para testes, coleta e análise de dados, e apresentação dos resultados.

#### 3.1 Objetivos Gerais

Analisar comparativamente o desempenho dos algoritmos de pathfinding como A\*, JPS e Theta\* em ambientes tridimensionais, considerando métricas de execução, concorrência e técnicas de otimização.

#### 3.2 Objetivos Específicos

1. Implementar uma estrutura de dados eficiente para representação de grafos direcionados e não direcionados voltada à execução de algoritmos de pathfinding.
2. Desenvolver uma infraestrutura de renderização para visualização de malhas tridimensionais e execução integrada dos algoritmos de pathfinding.
3. Implementar e adaptar algoritmos heurísticos (A\*, JPS, Theta\* e outros) para ambientes tridimensionais.
4. Adaptar a arquitetura do sistema para execução concorrente e realização de testes de estresse utilizando multithreading.
5. Desenvolver geradores de malhas topológicas complexas para simulação de diferentes cenários de teste.
6. Coletar e analisar métricas de desempenho dos algoritmos de pathfinding, incluindo tempo de execução, uso de memória e qualidade dos caminhos gerados.
7. Elaborar análises estatísticas e visuais comparativas dos resultados obtidos nos testes realizados.

## 4 FUNDAMENTAÇÃO TEÓRICA

Para a realização deste estudo, foi necessário adquirir conhecimentos em diversas áreas, incluindo algoritmos de busca, estruturas de dados, computação gráfica e concorrência. A seguir, são detalhados os principais tópicos que compõem a base teórica deste trabalho.

### 4.1 Pilha Tecnológica

#### 4.1.1 Linguagem de Programação

Todas as implementações e experimentos deste estudo foram realizados utilizando a linguagem de programação C++20. Escolhida por sua eficiência, controle de baixo nível e ampla adoção na indústria de jogos e simulações, a linguagem C++ oferece recursos avançados para manipulação de memória, concorrência e otimização de desempenho, essenciais para o desenvolvimento de algoritmos de pathfinding em ambientes tridimensionais.

A escolha do C++20 também se justifica pela disponibilidade de bibliotecas e *frameworks* que facilitam a implementação de estruturas de dados complexas, renderização gráfica e multithreading, além de permitir uma integração eficiente com APIs gráficas como OpenGL.

Enquanto a implementação da infraestrutura gráfica e dos algoritmos de pathfinding foi realizada do zero, a escolha do C++20 garantiu um ambiente de desenvolvimento robusto e flexível, permitindo a aplicação de técnicas avançadas de otimização e controle total sobre os recursos computacionais utilizados durante os testes.

##### 4.1.1.1 Estruturas de Dados

A implementação de algoritmos de pathfinding em malhas 3D requer o uso de estruturas de dados eficientes para representar grafos, filas de prioridade e outras estruturas auxiliares. Para isso, foi utilizada a biblioteca padrão do C++20, a STL (Standard Template Library), que oferece uma variedade de contêineres e algoritmos otimizados para manipulação de dados.

A escolha de estruturas de dados adequadas é crucial para garantir o desempenho dos algoritmos, especialmente em ambientes tridimensionais onde a complexidade pode aumentar significativamente. Por isso foram utilizadas estruturas como vetores, mapas de *hash* e filas de prioridade para representar os grafos e gerenciar os nós durante a execução dos algoritmos de busca.

##### 4.1.1.2 Funções *Lambda*

A introdução de funções *lambda* em C++11 e suas melhorias contínuas nas versões subsequentes, incluindo C++20, proporcionaram uma maneira mais concisa

de definir funções anônimas. No contexto deste estudo, as funções *lambda* foram amplamente utilizadas.

Por definição, uma função anônima é uma função “sem nome”, que pode ser definida e utilizada diretamente no local onde é necessária. Essas funções são particularmente úteis para operações que exigem uma função de curto prazo, como a definição de heurísticas em algoritmos de busca ou a implementação de funções de retorno (*callbacks*) para eventos específicos durante a renderização ou execução dos algoritmos.

#### 4.1.1.3 Biblioteca *jthread*

Um dos principais motivadores para o uso da versão C++20 é a introdução da biblioteca *jthread*, que simplifica significativamente a implementação de *multithreading* e concorrência. A classe *jthread* oferece uma interface mais segura e fácil de usar para gerenciamento de *threads*, incluindo a capacidade de interromper *threads* de forma cooperativa, o que é crucial para a realização de testes de estresse e simulações em ambientes dinâmicos.

Esta biblioteca utiliza dos princípios de RAII (*Resource Acquisition Is Initialization*) para garantir que os recursos sejam gerenciados de forma eficiente e segura, evitando problemas comuns em *multithreading*, como o encerramento abrupto do programa devido a *threads* não finalizadas ao saírem de escopo. A classe de *thread* é gerenciada de uma forma que, perante sua destruição, a *thread* faz uma junção (*join*) automaticamente, garantindo que os recursos sejam liberados corretamente. Isso não só facilita a implementação de concorrência, mas também melhora a segurança do código.

Uma junção (*join*) é uma operação que bloqueia a execução da *thread* chamadora até que a *thread* alvo termine sua execução. No caso da *jthread*, o *join* é automático, o que significa que quando um objeto de *jthread* é destruído, ele automaticamente chama o método *join()*, garantindo que a *thread* associada seja finalizada corretamente antes de liberar os recursos.

Também, a introdução dos tokens de parada (*stop tokens*) na biblioteca permite que as *threads* sejam interrompidas de maneira cooperativa, o que é especialmente útil para simulações e testes de estresse, onde é necessário controlar o tempo de execução das *threads* e garantir que elas possam ser finalizadas de forma segura.

#### 4.1.1.4 Testes Unitários e Google Test (GTest)

Para assegurar a corretude lógica e a robustez das bibliotecas desenvolvidas (tanto o motor gráfico IFCG quanto a biblioteca de grafos), incorporou-se a metodologia de testes unitários automatizados. A validação contínua de rotinas críticas do sistema é realizada utilizando o *framework* Google Test (GTest) (Google LLC, 2026). Esse arcabouço de testes está integrado diretamente ao ciclo de compilação do CMake (*tests/CMakeLists.txt*), permitindo que a integridade estrutural e de algoritmos do sistema

seja verificada a cada alteração da base de código, o que minimiza a ocorrência de regressões decorrentes de refinamentos ou novas otimizações.

#### 4.1.1.5 Bibliotecas Auxiliares (GLFW e GLM)

Adicionalmente, a pilha tecnológica conta com bibliotecas consolidadas para viabilizar a visualização gráfica e os cálculos espaciais. Utiliza-se a biblioteca GLFW (Vries, 2014) para gerenciar a criação da janela de exibição e o contexto de renderização do OpenGL, além de processar eventos de periféricos (teclado e mouse). Para as operações matemáticas de geometria analítica e álgebra linear no espaço tridimensional — essenciais no cálculo de heurísticas espaciais e transformações de vértices de malhas —, emprega-se a biblioteca GLM (OpenGL Mathematics), que implementa estruturas otimizadas de vetores e matrizes compatíveis com as especificações de sombreado do OpenGL (GLSL).

#### 4.1.2 Arquitetura de Programas

A arquitetura das bibliotecas e programas desenvolvidos para este estudo foi projetada para ser modular e escalável, permitindo a fácil integração de novos algoritmos de *pathfinding* e a adaptação a diferentes cenários de teste. Por isso foi escolhida uma arquitetura orientada a objetos, que facilita a organização do código e a reutilização de componentes.

Essa escolha se dá pela necessidade de criar uma infraestrutura gráfica personalizada e uma estrutura de dados eficiente para representar grafos, além de implementar algoritmos de *pathfinding* que possam ser facilmente adaptados e otimizados para ambientes tridimensionais. A arquitetura orientada a objetos permite encapsular a complexidade dos diferentes componentes do sistema, como a renderização gráfica, a representação de grafos e a execução dos algoritmos, facilitando a manutenção e evolução do código ao longo do desenvolvimento do estudo.

Em adição, o uso de uma arquitetura orientada a objetos permite abstrações de hierarquia e polimorfismo, que serão amplamente utilizadas na implementação de ambas as principais bibliotecas, tanto a de grafos quanto a de renderização gráfica.

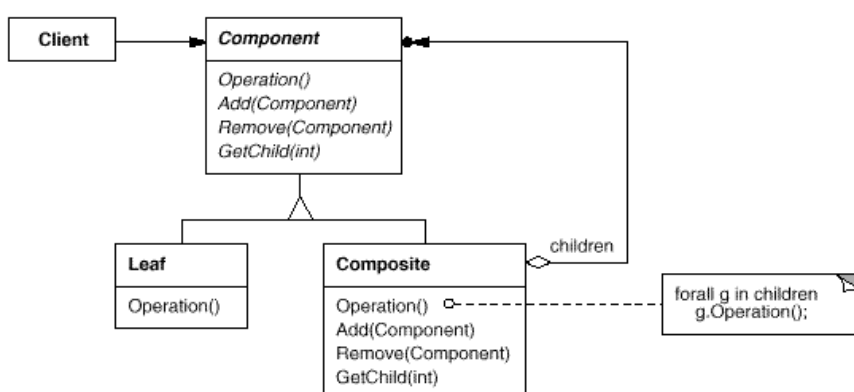
##### 4.1.2.1 Padrões de Projeto

Padrões de projeto são soluções reutilizáveis para problemas comuns de design de *software* (Gamma et al., 1994). No desenvolvimento deste estudo, foram aplicados diversos padrões de projeto para garantir a modularidade, flexibilidade e manutenibilidade do código.

O padrão *Composite* (Gamma et al., 1994) foi utilizado para representar a hierarquia de objetos na infraestrutura gráfica, permitindo que objetos complexos sejam tratados de forma uniforme, e na representação de grafos, onde grafos direcionados e não direcionados podem ser manipulados de maneira consistente. Neste padrão, objetos compostos (*Composite*) e objetos individuais (*Leaves* ou folhas) são tratados

de forma uniforme, com a introdução de uma superclasse comum (*Component*) que define a interface para ambos. Com isso, é possível que a classe *Composite* delegue a execução de uma função comum entre eles para todos os seus filhos, sejam eles *Composite* ou *Leaves*.

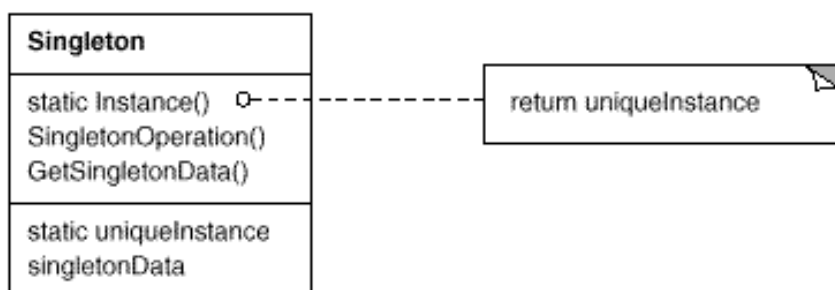
**Figura 1 – Diagrama de classes do padrão *Composite***



Fonte: Gamma et al. (1994)

Adicionalmente, foi utilizado o padrão *Singleton* (Gamma et al., 1994). Este padrão dita que uma classe deve ter apenas uma instância e fornecer um ponto global de acesso a essa instância. Para isso, o construtor da classe é privado e ela contém uma variável estática que armazena a única instância da classe (inicializada como nula). A classe também tem um método público estático que retorna a instância, criando-a se ainda não existir. Este padrão é útil para gerenciar recursos compartilhados, como o contexto gráfico do OpenGL, garantindo que haja um controle centralizado sobre a renderização e os recursos gráficos utilizados durante os testes.

**Figura 2 – Diagrama de classes do padrão *Singleton***



Fonte: Gamma et al. (1994)



## 4.2 Teoria dos Grafos

Uma vez que os algoritmos de *pathfinding* operam sobre estruturas de grafos para encontrar caminhos entre nós, o estudo da teoria dos grafos é a base fundamental deste experimento. A teoria dos grafos fornece as ferramentas e conceitos necessários para representar e manipular essas estruturas, permitindo a implementação eficiente dos algoritmos de busca.

Segundo Gomes (2022), um grafo é uma estrutura de dados composta por um conjunto de vértices (ou nós) e um conjunto de arestas (ou ligações) que conectam esses vértices. Os grafos podem ser direcionados, onde as arestas têm uma direção específica, ou não direcionados, onde as arestas não possuem direção. A representação de grafos pode ser feita de diversas formas, como listas de adjacência, matrizes de adjacência ou listas de arestas, cada uma com suas vantagens e desvantagens em termos de eficiência e uso de memória.

### 4.2.1 Definição e Propriedades dos Grafos

Um grafo é uma forma de representar conexões entre diferentes pontos. Basicamente, ele é composto por dois elementos principais: um conjunto de **vértices** (também chamados de nós), que são os pontos em si, e um conjunto de **arestas** (ou ligações), que são as linhas que conectam esses vértices. No contexto deste estudo, cada vértice pode representar uma posição ou uma célula em uma malha 3D, enquanto as arestas indicam os caminhos possíveis entre essas posições.

As arestas podem ter **pesos**, que são valores numéricos associados a elas. Esse peso pode indicar o “custo” de percorrer aquela ligação, como a distância física, o tempo gasto, ou a dificuldade de travessia (por exemplo, subir uma colina íngreme). O conceito de **adjacência** se refere a vértices que estão diretamente conectados por uma aresta, e um **caminho** é uma sequência de vértices adjacentes que leva de um ponto inicial a um destino (Gomes, 2022). Essas propriedades são fundamentais para os algoritmos de busca que visam encontrar as melhores rotas.

### 4.2.2 Grafos Direcionados e Não Direcionados

A natureza das arestas em um grafo define se ele é **direcionado** ou **não direcionado**. Em um grafo **não direcionado**, se existe uma aresta entre dois vértices A e B, significa que o caminho pode ser percorrido tanto de A para B quanto de B para A, com o mesmo custo (ou seja, a aresta (A, B) é idêntica à aresta (B, A)). Pense em uma estrada de mão dupla em um terreno plano.

Já em um grafo **direcionado**, a aresta tem um sentido específico. Uma aresta de A para B não implica necessariamente que existe uma aresta de B para A, ou que o custo de retorno seja o mesmo. Isso é particularmente relevante em ambientes 3D, onde o custo de subir uma rampa íngreme pode ser muito diferente (e maior) do que descer a mesma rampa, ou até mesmo haver obstáculos que permitem passagem em apenas um sentido.

#### 4.2.3 MST (*Minimum Spanning Tree*)

Uma *Minimum Spanning Tree* (MST), ou Árvore Geradora Mínima, é um sub-grafo de um grafo original que conecta todos os vértices do grafo sem formar nenhum ciclo, e cuja soma dos pesos de todas as suas arestas é a menor possível (Gomes, 2022). Em outras palavras, é a forma mais “econômica” de conectar todos os pontos.

Para o propósito deste trabalho, algoritmos de construção de MST, como o algoritmo de Kruskal, são de grande utilidade para a **geração procedural de labirintos**. Ao aplicar esses algoritmos sobre uma grade de possíveis conexões (representando as células do labirinto como vértices e as paredes como arestas com pesos aleatórios), é possível criar labirintos “perfeitos”, onde existe um único caminho entre quaisquer dois pontos. Esses labirintos servem como cenários de teste complexos e controláveis para avaliar o desempenho dos diferentes algoritmos de pathfinding em ambientes 3D.

### 4.3 Algoritmos de Busca Heurística

Na área de estudo de grafos, existe uma grande concentração de pesquisas em algoritmos de busca heurística, que são técnicas que utilizam informações adicionais (heurísticas) para guiar a busca por um caminho mais eficiente no grafo. Esses algoritmos são amplamente utilizados em jogos e simulações para encontrar rotas entre pontos em um ambiente tridimensional.

#### 4.3.1 Dijkstra

O algoritmo de Dijkstra é um dos métodos mais fundamentais para encontrar o caminho mais curto entre dois vértices em um grafo com arestas de pesos não negativos. Segundo Gomes (2022), o algoritmo funciona explorando sistematicamente todos os caminhos possíveis a partir de um vértice inicial, mantendo uma lista de distâncias mínimas conhecidas para cada nó. Ele utiliza uma fila de prioridade para sempre expandir o nó com a menor distância acumulada, garantindo que, ao chegar no destino, o caminho encontrado seja de fato o mais curto. Por sua natureza exaustiva, o Dijkstra é garantidamente ótimo, mas pode ser computacionalmente custoso em grafos muito grandes, pois “olha para todos os lados” antes de decidir a direção final.

#### 4.3.2 Heurísticas

Diferente do Dijkstra, que explora o grafo de forma cega, a busca heurística utiliza informações extras para tentar “adivinhar” qual direção é mais promissora. Uma heurística é, essencialmente, uma estimativa do custo restante para chegar ao destino (Hart; Nilsson; Raphael, 1968).

Um exemplo clássico é a **Distância Euclidiana** (a linha reta entre dois pontos). Em um mapa 3D, se soubermos as coordenadas do ponto atual  $(x_1, y_1, z_1)$  e do destino  $(x_2, y_2, z_2)$ , podemos calcular a distância direta entre eles. Embora essa distância ignore obstáculos como paredes ou montanhas, ela serve como um guia excelente

para o algoritmo priorizar nós que estão fisicamente mais próximos do objetivo, reduzindo drasticamente o número de vértices que precisam ser analisados.

#### 4.3.3 A\*

O algoritmo A\* (A-Star) é uma evolução do Dijkstra que incorpora o uso de heurísticas para aumentar a eficiência da busca. Ele avalia cada nó utilizando uma função de custo  $f(n) = g(n) + h(n)$ , onde:

- $g(n)$  é o custo real acumulado do ponto de partida até o nó atual  $n$ .
- $h(n)$  é o valor da heurística (a estimativa do custo de  $n$  até o destino).

Ao combinar o progresso real com a estimativa futura, o A\* consegue focar a busca na direção do objetivo, evitando explorar áreas do grafo que claramente não levam ao caminho mais curto (Hart; Nilsson; Raphael, 1968). Se a heurística utilizada for “admissível” (ou seja, nunca superestimar o custo real), o A\* garante encontrar o caminho ótimo, unindo a precisão do Dijkstra com a velocidade de uma busca direcionada.

#### 4.3.4 Jump Point Search (JPS)

O *Jump Point Search* (JPS) é uma otimização do algoritmo A\* específica para grades uniformes (como as malhas de navegação). Em vez de analisar cada vizinho imediato de um nó (passo a passo), o JPS “pula” grandes áreas vazias do grafo que não oferecem mudanças de direção interessantes (Nobes et al., 2022).

Ele identifica pontos críticos chamados *Jump Points* — locais onde a presença de um obstáculo força o algoritmo a considerar uma nova direção. Ao saltar diretamente entre esses pontos, o JPS reduz significativamente o número de operações na fila de prioridade e o uso de memória, sendo frequentemente ordens de grandeza mais rápido que o A\* tradicional em ambientes com grandes espaços abertos, mantendo a mesma garantia de encontrar o caminho mais curto.

#### 4.3.5 Theta\*

O algoritmo Theta\* é uma extensão do A\* que permite a busca de caminhos mais diretos em ambientes tridimensionais, ao invés de se limitar a movimentos ortogonais ou diagonais em uma grade. Ele combina a eficiência do A\* com a capacidade de “ver” através de obstáculos, permitindo que o caminho seja ajustado dinamicamente para evitar curvas desnecessárias (Nash; Koenig, 2013).

### 4.4 Computação Gráfica

Por causa da necessidade de controle total sobre a infraestrutura gráfica e a implementação dos algoritmos, optou-se por desenvolver um motor gráfico do zero, denominado IFCG<sup>1</sup> (Instituto Federal Catarinense/Computação Gráfica). Esta decisão

---

<sup>1</sup>Disponível em: <https://github.com/andrevbastos/IFCG>. Acesso em: 22 mai. 2026.

foi motivada pela necessidade de um ambiente de teste personalizado, que permita a coleta de dados em condições controladas e a aplicação de otimizações específicas para os algoritmos de pathfinding.

#### 4.4.1 OpenGL API

O OpenGL é uma API de gráficos 3D amplamente utilizada para renderização de gráficos em tempo real (Vries, 2014). No desenvolvimento da infraestrutura gráfica para este estudo, o OpenGL foi escolhido por sua flexibilidade, desempenho e ampla adoção na indústria de jogos e simulações. A utilização do OpenGL também facilita a implementação de técnicas avançadas de renderização e otimização, garantindo que os testes sejam realizados em um ambiente gráfico realista.

##### 4.4.1.1 Malhas

As malhas são uma representação comum de superfícies em computação gráfica, onde uma superfície é representada por um conjunto de vértices conectados por arestas e faces. No contexto deste estudo, as malhas serão utilizadas para representar os cenários de teste em que os algoritmos de pathfinding serão executados.

Cada conjunto de vértices e arestas diferentes são armazenados em *buffers* denominados VBO (Vertex Buffer Object) e EBO (Element Buffer Object), respectivamente, que são gerenciados pela GPU para renderização eficiente. Cada um desses *buffers* é associado a um VAO (Vertex Array Object), que encapsula o estado necessário para renderizar a malha, incluindo as ligações dos *buffers* e as configurações de atributos de vértice.

O conhecimento detalhado do funcionamento de cada buffer se prova essencial, quando consideramos a forma que o OpenGL lida com a renderização de malhas e troca de dados entre CPU e GPU. Uma vez que o OpenGL tem grandes problemas para lidar com múltiplas *threads*, o gerenciamento eficiente dos *buffers* e a minimização de operações de troca de dados entre CPU e GPU são cruciais para garantir o desempenho dos algoritmos de pathfinding em ambientes tridimensionais.

##### 4.4.1.2 Programação Orientada a Eventos e Funções de Retorno (*Callback*)

Embora o OpenGL seja estritamente uma API de renderização, sem conhecimento nativo sobre o sistema operacional, janelas ou periféricos de entrada, a infraestrutura gráfica desenvolvida (IFCG) utiliza a biblioteca GLFW para o gerenciamento da janela e a criação do contexto gráfico. Essa integração permite implementar um modelo de programação orientada a eventos, onde o fluxo de execução é guiado por interações externas, como atualizações do sistema e entradas do usuário. As funções de retorno (*callback*) fornecidas pela API do GLFW foram implementadas na camada de gerenciamento do motor para interceptar eventos de teclado e mouse, servindo como uma ponte de comunicação interativa com o sistema de renderização.

#### 4.4.1.3 A Máquina de Estados e o Contexto OpenGL

Para lidar com a renderização em si, o OpenGL opera como uma vasta máquina de estados. Todas as configurações e referências de dados ficam armazenadas no que é chamado de Contexto OpenGL. Dessa forma, para renderizar uma malha, é necessário configurar o estado da máquina de acordo com as características do objeto — como a vinculação dos *buffers* VBO e EBO — antes de emitir os comandos de desenho para a GPU. Como mudanças excessivas de estado geram alto custo de processamento (*overhead*), o gerenciamento eficiente dos *buffers* procura minimizar as trocas de estado e agrupar comandos sempre que possível, otimizando o tráfego de dados entre a CPU e a GPU.

#### 4.4.1.4 Concorrência e Isolamento de *Threads*

Essa arquitetura baseada em contexto de estado impõe restrições rígidas à implementação de concorrência. O contexto do OpenGL é estritamente atrelado à *thread* em que foi ativado, tipicamente a *thread* principal. Tentativas de acessar ou modificar o estado da API gráfica a partir de múltiplas *threads* simultaneamente causam violações de acesso à memória, resultando em falhas críticas de limite de endereço, como erros SIGSEGV. Por isso, a arquitetura deste trabalho isola a renderização na *main thread* e delega o processamento pesado de dados para as *worker threads* do TaskMaster.

### 4.5 Geração de Cenários de Teste

Para garantir a diversidade e complexidade dos cenários de teste, é necessário implementar algoritmos de geração procedural de terrenos. Esses algoritmos permitem criar malhas 3D complexas e variadas, simulando diferentes tipos de ambientes que os algoritmos de pathfinding podem encontrar em situações reais.

A geração procedural é uma técnica utilizada para criar conteúdo de forma automática, utilizando algoritmos que produzem resultados variados e complexos a partir de um conjunto de regras ou parâmetros. No contexto deste estudo, a geração procedural será aplicada para criar terrenos que servirão como cenários de teste para os algoritmos de pathfinding.

#### 4.5.1 Labirintos

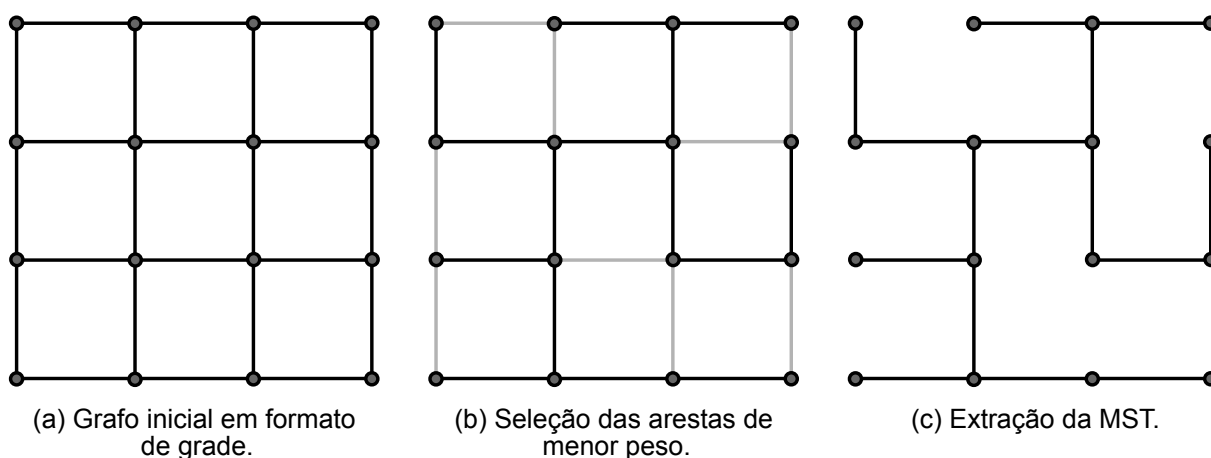
Temos como definição que labirintos são estruturas complexas compostas por caminhos interconectados, onde o objetivo é encontrar uma rota do ponto de partida até um destino específico. A geração de labirintos pode ser realizada utilizando diversos algoritmos. Para este estudo, foi utilizado o algoritmo de Kruskal (Gomes, 2022), que é um método eficiente para gerar labirintos perfeitos, ou seja, labirintos sem ciclos, onde existe apenas um caminho entre quaisquer dois pontos.

Inicialmente, para gerar labirintos com o algoritmo de Kruskal, é construído um grafo em formato de grade (Figura 3 (a)). Neste grafo, cada célula do labirinto é

representada por um vértice, e as possíveis conexões entre as células são representadas por arestas de pesos aleatórios.

O algoritmo de Kruskal é então aplicado para construir uma *Minimum Spanning Tree* (MST), ou árvore geradora mínima, a partir desse grafo, selecionando as arestas de menor peso e garantindo que não sejam formados ciclos (Figura 3 (b)). O resultado é um labirinto perfeito, onde cada célula tem um caminho que a conecta a qualquer outra célula, criando um caminho único entre o ponto de partida e o destino (Figura 3 (c)).

**Figura 3 – Etapas de geração de um labirinto perfeito utilizando o algoritmo de Kruskal**



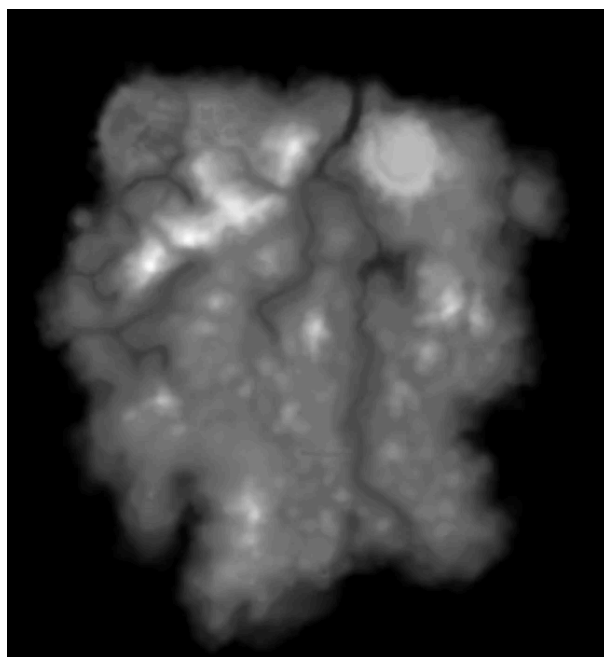
Fonte: Elaborado pelo autor (2026).

Com um labirinto perfeito em mãos, é possível sobrepor múltiplos labirintos perfeitos para criar cenários mais complexos. Essa técnica de sobreposição permite aumentar a densidade de obstáculos e criar topologias mais desafiadoras para os algoritmos de pathfinding, simulando ambientes mais realistas e variados.

#### 4.5.2 Mapas de Altura

Um mapa de altura nada mais é do que uma representação gráfica de um terreno, onde cada pixel da imagem representa uma coordenada no espaço tridimensional e a intensidade dos valores de cor indica a elevação do terreno naquela coordenada. Diversos desenvolvedores de jogos e simulações utilizam mapas de altura para criar terrenos realistas, como montanhas, vales e planícies. Oferecendo, na internet, uma variedade de cenários para testar os algoritmos de *pathfinding*.

**Figura 4 – Exemplo de mapa de altura**



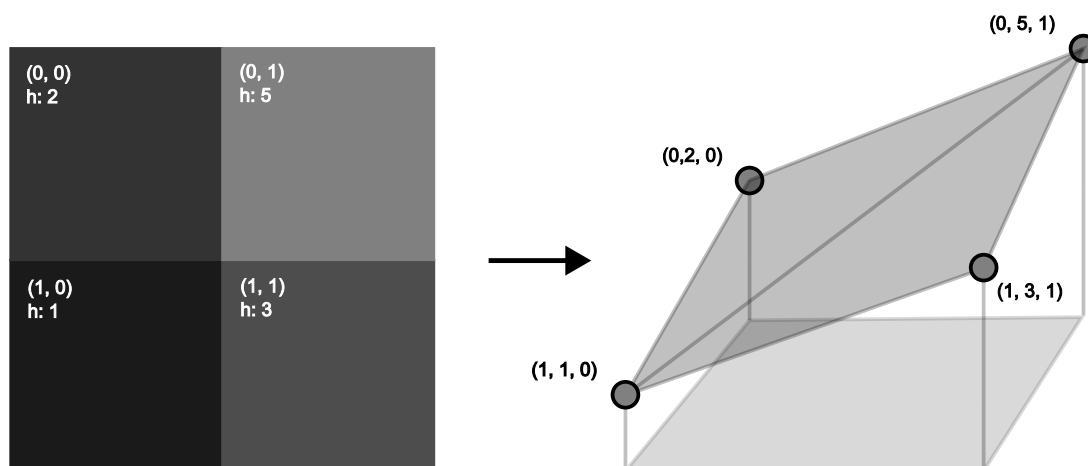
Fonte: The Imperial Library (2026).

Com isso, é possível gerar malhas 3D complexas a partir de simples imagens. Essa técnica é amplamente utilizada em jogos e simulações para criar ambientes naturais, como montanhas, vales e planícies, oferecendo uma variedade de cenários para testar os algoritmos de pathfinding.

É possível encontrar diversos mapas de altura gratuitos na internet, que podem ser utilizados para gerar malhas 3D realistas e complexas. Esses mapas de altura podem ser processados para extrair a geometria do terreno, criando uma malha que pode ser utilizada como cenário de teste para os algoritmos de pathfinding.

Para poder criar uma malha a partir dessas imagens, primeiro é necessário processar a imagem do mapa de altura para extrair as coordenadas dos vértices e suas respectivas alturas. Considerando  $i$  e  $j$  como os índices dos pixels da imagem,  $h$  como o valor de intensidade da cor do pixel e  $(x, y, z)$  como as coordenadas 3D de cada vértice da malha, é possível atribuir as coordenadas de cada vértice da malha com  $(x, y, z) = (i, h, j)$ .

**Figura 5 – Conversão de um mapa de altura para uma malha 3D**



Fonte: Elaborado pelo autor.

É importante ressaltar que os valores de cor  $(r, g, b)$  de uma imagem em escala de cinza variam igualmente de 0 a 255. Para evitar que as variações de altura no cenário 3D sejam desproporcionais às distâncias horizontais, o valor do pixel precisa ser normalizado e escalonado. Considerando  $v$  como o valor  $r, g$  ou  $b$  do pixel e  $H$  como a altura máxima desejada para o terreno, a elevação  $h$  de cada vértice é calculada pela fórmula:

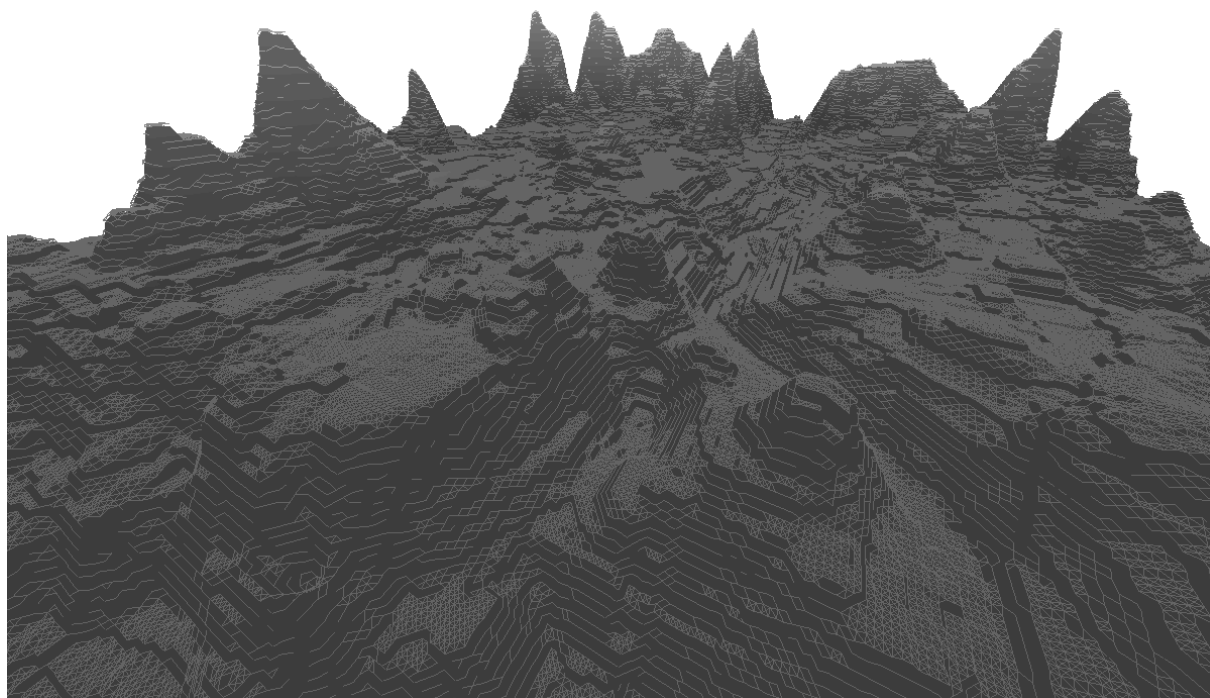
$$h = \left( \frac{v}{255} \right) * H$$

Dessa forma, um pixel totalmente preto ( $v = 0$ ) resultará em uma altura de 0, enquanto um pixel totalmente branco ( $v = 255$ ) atingirá a altura máxima estipulada ( $H$ ), permitindo um controle preciso sobre a amplitude do relevo gerado.

Por fim, são criadas faces conectando cada vértice com seus vizinhos diretos, formando uma malha de triângulos que representa a superfície do terreno. Essa malha pode então ser utilizada como cenário de teste para os algoritmos de pathfinding, permitindo avaliar seu desempenho em ambientes tridimensionais complexos.



**Figura 6 – Exemplo de malha gerada a partir de um mapa de altura**



Fonte: Elaborado pelo autor.

#### 4.5.3 Geração de Ruídos

Utilizar mapas de altura foi uma grande vantagem para a geração de cenários realistas, mas é ineficiente procurar imagens pré-existentes para cada cenário de teste que se deseja criar. Para isso, é indispensável a implementação de algoritmos de geração de ruídos.

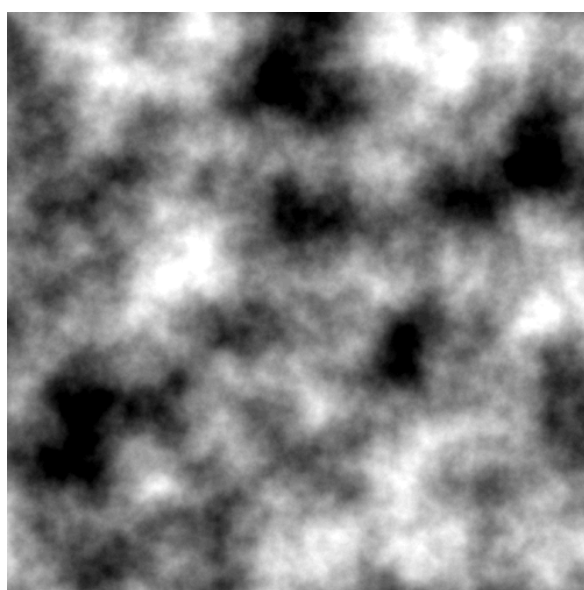
Um ruído é uma função matemática que gera valores pseudoaleatórios, mas de forma controlada, para criar padrões que se assemelham a fenômenos naturais. Existem diversos tipos de ruídos, como o ruído Perlin, o ruído Simplex e o ruído de valor, cada um com suas características e aplicações específicas. Porém, para essa pesquisa, o foco será no ruído Perlin, devido à sua capacidade de gerar padrões suaves e naturais, ideal para simular terrenos realistas.

O ruído de Perlin é um algoritmo de geração de ruído gradiente procedural que é amplamente utilizado para criar texturas e terrenos realistas em gráficos 3D. Ele foi desenvolvido por Perlin (1985) e é conhecido por produzir padrões de ruído suaves e naturais (Figura 7 (a)), o que o torna ideal para simular superfícies como montanhas, nuvens e oceanos. Com isso em mente, podemos utilizar desse algoritmo para gerar nossos próprios mapas de altura, sem precisar recorrer a imagens pré-existentes. Isso nos dá controle total sobre a geração dos cenários de teste,

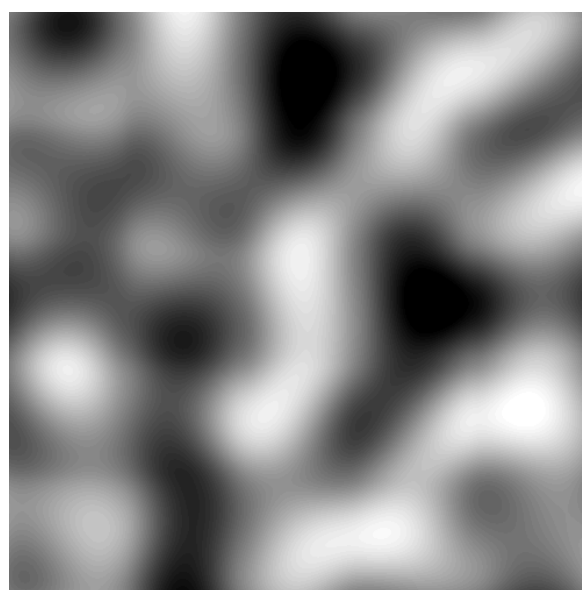
permitindo criar uma variedade de terrenos com diferentes características e desafios para os algoritmos de *pathfinding*.

Para conseguir um ruído suave e natural, o algoritmo gera diversas camadas de texturas diferentes (Figura 7 (b)) e as sobrepõe, essas camadas são chamadas de oitavas (*octaves*). Ao sobrepor múltiplas oitavas, é usada uma função de interpolação, que será detalhada na Seção 4.4.3.1, para suavizar a transição entre os valores gerados por cada camada, criando um resultado final que se assemelha a padrões naturais. A combinação de múltiplas oitavas permite criar terrenos com detalhes variados, desde grandes elevações até pequenas variações de altura.

**Figura 7 – Exemplo de ruído de Perlin e uma de suas oitavas (*octaves*)**



(a) Ruído de Perlin completo.

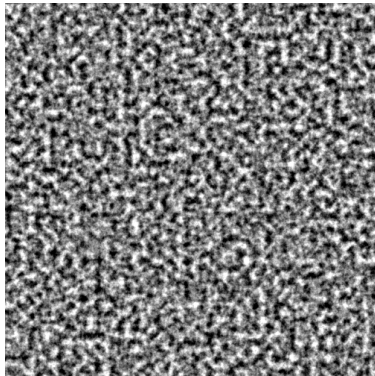


(b) Uma *octave* do ruído de Perlin.

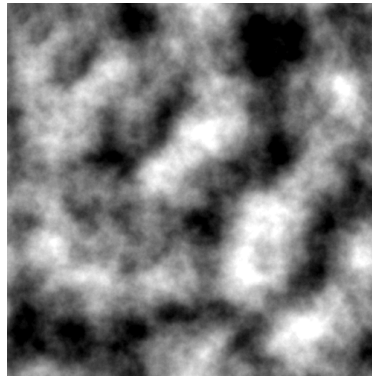
Fonte: Elaborado pelo autor.

Para gerar uma *octave* do ruído de Perlin, primeiro é necessário criar uma grade de tamanho unitário com vetores de gradiente unitários em cada ponto de interseção da grade, onde cada vetor aponta em uma direção aleatória. O tamanho de cada célula da grade determina o nível de detalhe do ruído, onde células maiores produzem um ruído mais suave, enquanto células menores geram um ruído mais detalhado.

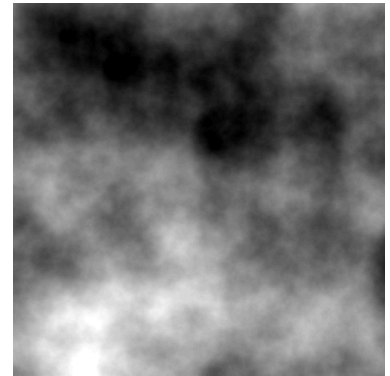
**Figura 8 – Ruídos de Perlin com diferentes tamanhos de células**



(a) Ruído gerado com células pequenas.



(b) Ruído gerado com células medianas.



(c) Ruído gerado com células grandes.

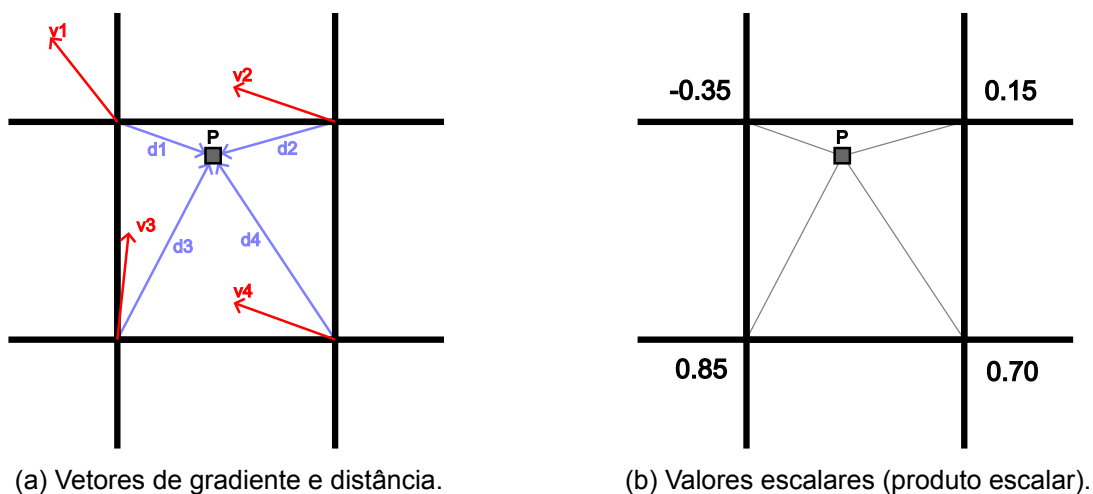
Fonte: Elaborado pelo autor.

Para cada ponto no espaço (ou *pixel*), o algoritmo calcula os vetores de distância entre ele e cada ponto de interseção de sua respectiva célula da grade. Em seguida, é calculado o produto escalar entre os vetores de distância  $\vec{a} = (a_x, a_y)$  e os vetores de gradiente correspondentes  $\vec{b} = (b_x, b_y)$ , pela fórmula:

$$\vec{a} \cdot \vec{b} = (a_x * b_x) + (a_y * b_y)$$

O resultado do produto escalar é um valor numérico que representa a contribuição do vetor de gradiente para o *pixel* em questão. Por fim, é aplicada uma função de interpolação (Seção 4.4.3.1), onde cada valor escalar é suavizado para criar uma transição entre o *pixel* de forma ponderada, ou seja quanto mais próximo o *pixel* estiver do ponto de interseção, maior será a contribuição do vetor de gradiente para o valor final do ruído.

**Figura 9 – Cálculo do produto escalar em uma célula do ruído de Perlin**

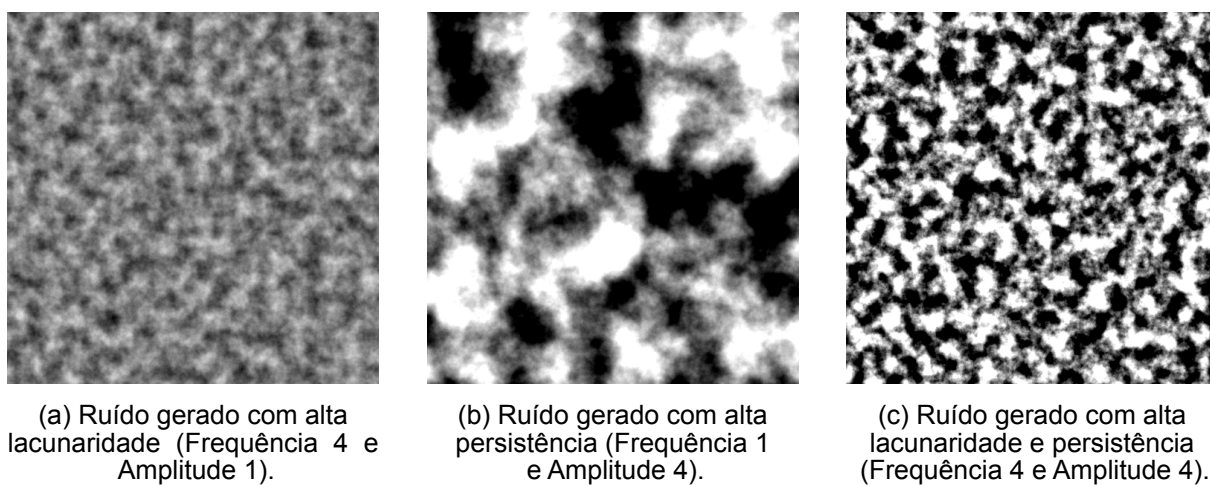


Fonte: Elaborado pelo autor.

Com uma oitava (*octave*) do ruído de Perlin gerada, é possível sobrepor múltiplas oitavas para criar um ruído mais complexo e detalhado. Porém, para evitar que as oitavas sejam idênticas, é necessário aplicar valores que alterem sua geração. Para isso, são introduzidos valores de frequência (lacunaridade) e amplitude (persistência), onde a frequência determina o número de detalhes presentes na oitava, e a amplitude controla a intensidade desses detalhes.

Conforme Patel (2015) explica, a combinação de múltiplas oitavas com diferentes frequências e amplitudes permite criar terrenos com uma variedade de características, desde grandes elevações até pequenas variações de altura, simulando ambientes naturais de forma realista.

**Figura 10 – Ruídos de Perlin com diferentes frequências e amplitudes**



Fonte: Elaborado pelo autor.

Assim, em cada ponto do espaço  $(x, y)$ , o valor base do ruído acumulado  $V(x, y)$  é definido pela soma ponderada das oitavas (*octaves*):

$$V(x, y) = \sum_{i=0}^{N-1} \left( a_i \cdot \text{perlin} \left( \frac{x \cdot f_i}{\lambda}, \frac{y \cdot f_i}{\lambda} \right) \right)$$

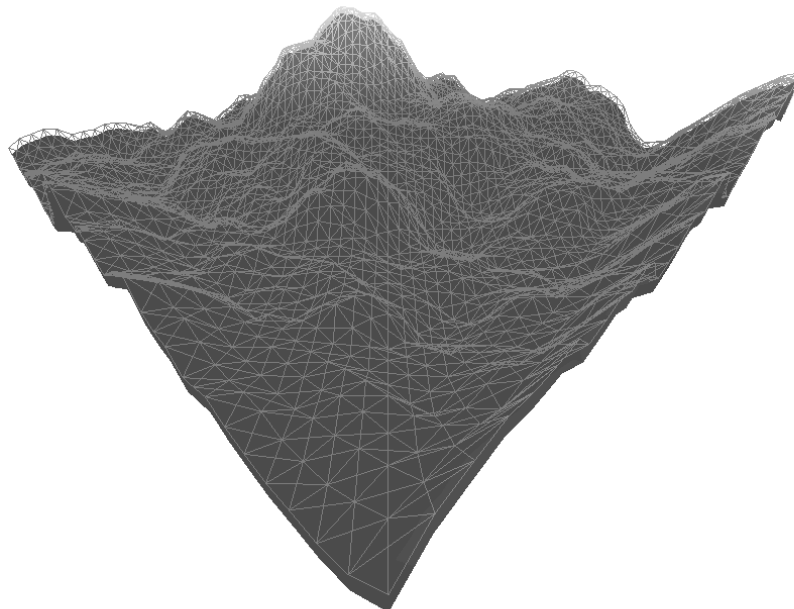
Onde  $N$  é o número de oitavas (*octaves*),  $\lambda$  é o tamanho da célula,  $f_i = F \cdot 2^i$  representa o aumento exponencial da frequência e  $a_i = A \cdot 0.5^i$  representa o decaimento exponencial da amplitude.

Após o acúmulo, o valor final  $V_f(x, y)$  do terreno é limitado para um intervalo de  $[-1, 1]$  e pode ser remapeado para um intervalo de  $[0, H]$ , onde  $H$  é a altura máxima desejada para o terreno, utilizando a fórmula:

$$V_f(x, y) = \left( \frac{V(x, y) + 1}{2} \right) \cdot H$$

Finalmente, o valor  $V_f(x, y)$  é utilizado para definir a elevação de cada vértice da malha como  $h$ . Isso permite gerar terrenos como feito anteriormente a partir de mapas de altura, mas com controle total sobre a geração dos cenários de teste, criando uma variedade de terrenos sem depender de imagens de terceiros.

**Figura 11 – Exemplo de malha gerada a partir de um ruído de Perlin**



Fonte: Elaborado pelo autor.

#### 4.5.3.1 Função de Interpolação

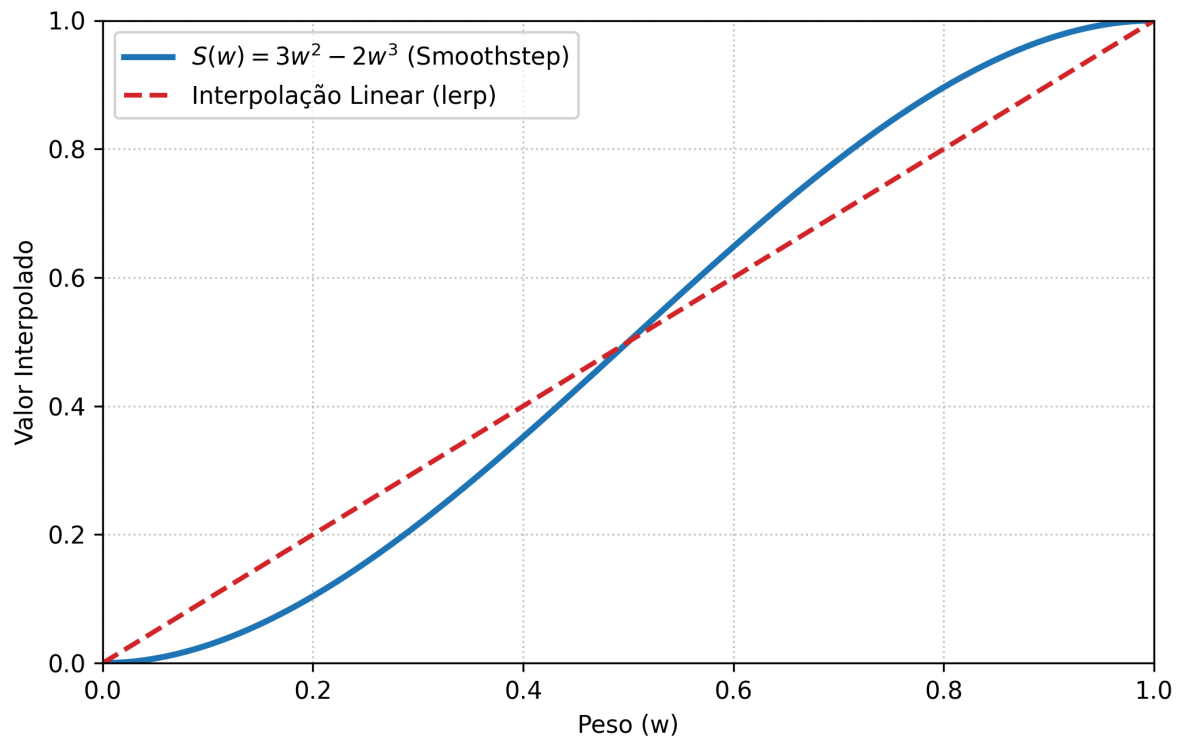
A função de interpolação é um componente crucial no algoritmo de geração de ruído de Perlin, pois é responsável por suavizar a transição entre os valores gerados por cada camada de ruído, criando um resultado final que se assemelha a padrões naturais.

A função de interpolação utilizada no ruído de Perlin é conhecida como função de suavização. Zipped (2023) ressalta que a interpolação linear simples não é adequada de forma isolada para gerar padrões naturais, pois deixa transições abruptas entre os valores de ruído, resultando em um terreno com bordas visíveis entre as células da grade.

Para solucionar esse problema e garantir uma transição orgânica, emprega-se uma curva de atenuação (ou função *fade*). No algoritmo original de Perlin, essa suavização é alcançada através de uma função polinomial cúbica, conhecida como *Smoothstep*, definida matematicamente por:

$$S(w) = 3w^2 - 2w^3$$

**Figura 12 – Função de Suavização no Ruído de Perlin**



Fonte: Elaborado pelo autor.

Essa curva mapeia o peso  $w$  (a posição relativa da coordenada, variando de 0 a 1) para um valor suavizado. A principal vantagem matemática do uso dessa função é que a sua primeira derivada é igual a zero em ambas as extremidades ( $w = 0$  e  $w = 1$ ). Isso garante que a transição de influência entre os gradientes de duas células vizinhas seja perfeitamente contínua, ocultando os limites da grade original.

Após obter o peso suavizado  $S(w)$ , o algoritmo realiza a interpolação linear padrão entre os valores de contribuição escalar  $a_0$  e  $a_1$  calculados:

$$f(a_0, a_1, w) = a_0 + (a_1 - a_0) \cdot S(w)$$

## 4.6 Concorrência e Paralelismo

A evolução do hardware moderno, com o aumento do número de núcleos de processamento, tornou o paralelismo uma ferramenta essencial para manter a responsividade em aplicações gráficas e intensivas em dados.

### 4.6.1 Multithreading com C++20

A linguagem C++20 introduziu várias melhorias para a programação concorrente, incluindo a classe `std::jthread`, que é uma extensão da classe `std::thread` com

suporte integrado para cancelamento de threads. O uso de `std::jthread` permite que as threads sejam encerradas de forma limpa e segura, aproveitando o padrão RAII (*Resource Acquisition Is Initialization*) para garantir que as threads sejam automaticamente unidas (*joined*) em chamadas de destruição.

Em seu construtor, as `std::jthread` recebem uma função lambda que encapsula a lógica de execução da thread trabalhadora. Funções lambda são funções anônimas que podem capturar variáveis do escopo onde foram definidas, facilitando a passagem de dados. Ao ser instanciada, a `std::jthread` inicia a execução da lambda em uma nova linha de processamento paralela.

#### 4.6.2 Tokens de Parada

O `std::stop_token` é um mecanismo que permite sinalizar a uma thread que ela deve parar sua execução de forma cooperativa. Isso é especialmente útil para evitar bloqueios e garantir encerramentos limpos. No seu laço (*loop*) interno, a função lambda pode verificar periodicamente o estado do `std::stop_token` para determinar se deve continuar executando ou encerrar a thread. Uma vantagem da `std::jthread` é a injeção automática desse token caso a função lambda possua um parâmetro correspondente, eliminando a necessidade de instanciá-lo explicitamente.

#### 4.6.3 Variáveis Condicionais

O `std::condition_variable_any` é uma ferramenta de sincronização que permite que as threads esperem por condições específicas, facilitando a coordenação entre threads trabalhadoras e a *main thread*. Quando as threads trabalhadoras são criadas, elas entram em estado de espera usando a variável condicional, aguardando que a *main thread* adicione uma tarefa à fila. O uso desta ferramenta é crucial para garantir que as tarefas sejam processadas em ordem e sem o uso excessivo de CPU por meio de *polling*.

#### 4.6.4 Monitores

Conforme explica Ladeira (2026), o padrão Monitor é uma implementação de alto nível para controle de sincronização. Ele encapsula tanto os dados quanto os métodos que operam sobre esses dados, utilizando mecanismos de bloqueio (*mutexes*) para garantir exclusão mútua e variáveis de condição para coordenar a execução das threads. A adoção do padrão Monitor visa prevenir condições de corrida (*race conditions*) entre as rotinas assíncronas de processamento e a *main thread*, garantindo a integridade dos recursos compartilhados como os dados da malha e do grafo.

#### 4.6.5 Task Scheduler

O *Task Scheduler* (Agendador de Tarefas) da aplicação centraliza o gerenciamento de todas as rotinas concorrentes que não envolvem renderização. O seu papel principal é evitar o travamento da thread principal (onde roda a interface e o contexto



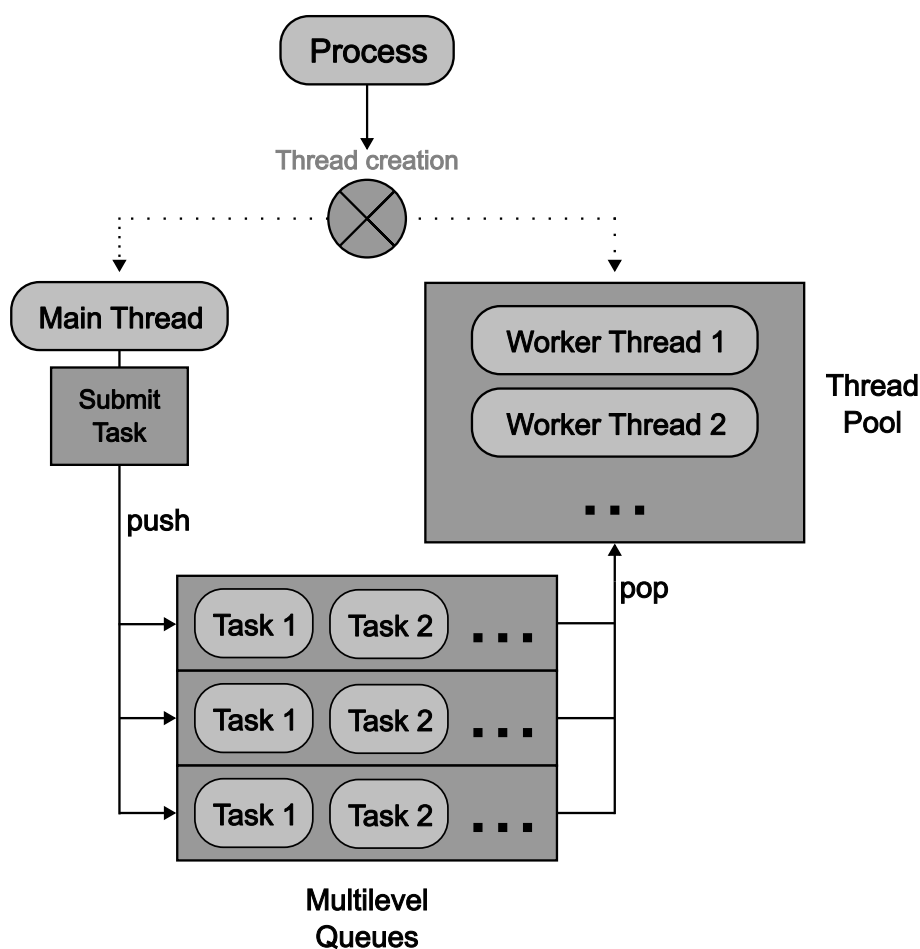
OpenGL) através da delegação de processamento intensivo para um pool de threads trabalhadoras que operam em paralelo (Williams, 2019). No contexto deste projeto, as tarefas delegadas incluem a modelagem de ruído de Perlin, a leitura e decodificação de mapas de altura, a triangulação física de malhas 3D e a posterior extração do grafo lógico necessário para o cálculo das rotas.

#### 4.6.6 Fila Multinível

Como as tarefas executadas pelo agendador possuem tempos de processamento e níveis de importância diferentes para o usuário, o sistema utiliza filas multinível ordenadas por prioridade.

- As tarefas de alta prioridade (`Priority::High`) são reservadas para etapas cruciais que desbloqueiam as fases seguintes do pipeline, como a definição dos parâmetros de criação e a geração dos ruídos.
- As tarefas de média prioridade (`Priority::Medium`) compreendem o processamento estrutural mais longo, que envolve a triangulação geométrica e a extração do grafo.
- As tarefas de baixa prioridade (`Priority::Low`) englobam processos de escrita em disco (I/O) utilizando a biblioteca `stb_image_write`, como o salvamento de imagens png dos mapas gerados. Por dependerem da latência física do sistema de armazenamento, essas operações rodam apenas quando não há tarefas computacionais prioritárias na fila, minimizando a contenção por barramento e memória.

Figura 13 – Diagrama de Thread Pool e filas multinível



Fonte: Elaborado pelo autor.

#### 4.6.7 Segurança de Memória e Compartilhamento de Estado

A execução simultânea de algoritmos de busca e rotinas de geração procedural em múltiplas threads operárias traz riscos críticos de corrupção de memória. Se uma thread trabalhadora tentar ler ou varrer os nós de um grafo (composto por objetos `Vertex3D` e vetores de adjacência) no exato instante em que a thread principal deleta, substitui ou recria o cenário tridimensional, ocorre um erro de ponteiro solto (dangling pointer) que causa o encerramento do programa por falha de segmentação (erro SIGSEGV).

Para evitar a ocorrência de condições de corrida (race conditions) e vazamento de recursos sem introduzir cópias pesadas de dados na memória RAM, a infraestrutura abandona o uso de ponteiros brutos na comunicação entre threads. Em vez disso, o grafo lógico é compartilhado por meio de ponteiros inteligentes do tipo `std::shared_ptr<common::lwGraph<Vertex3D>>`. Quando uma tarefa de busca de caminhos é inserida no `TaskMaster`, ela recebe uma cópia do `std::shared_ptr`, o que incrementa o contador de referências do recurso. Dessa maneira, mesmo que a thread

principal delete a representação visual da malha ou inicie uma nova simulação, o bloco de memória do grafo em execução permanece válido até que a thread operária termine seu processamento e libere o ponteiro, garantindo total isolamento e segurança. As referências constantes (como `const Vertex3D&`) são reservadas apenas para acessos de leitura locais dentro de escopos bem controlados onde a validade temporal do objeto é garantida por design.

#### **4.7 Análise Estatística**

O modelo ANOVA (*Analysis of Variance*) é uma técnica estatística utilizada para comparar as médias de diferentes grupos e determinar se existem diferenças significativas entre eles. Com base na literatura Domingues (2026), o ANOVA é particularmente útil quando se deseja avaliar o impacto de diferentes fatores em uma variável dependente, permitindo identificar quais fatores têm efeito significativo sobre os resultados.

Para confirmar o nível de influência de cada fator, usa-se o teste de Tukey, que é um método de comparação múltipla que permite identificar quais grupos diferem significativamente entre si. O teste de Tukey é aplicado após a realização do ANOVA para fornecer uma análise detalhada das diferenças entre os grupos.

## 5 TRABALHOS RELACIONADOS

Este capítulo apresenta uma análise comparativa detalhada entre a proposta deste trabalho e os principais estudos correlatos da literatura recente que realizam análises estatísticas e *benchmarks* de algoritmos de *pathfinding*.

### 5.1 Johansson (2024)

No trabalho intitulado *Adapting Pathfinding Algorithms for 3D Environments: Performance Analysis and Real-World Applications*, Johansson; Others (2024) realiza uma avaliação de desempenho em termos de tempo de execução e consumo de memória dos algoritmos Dijkstra e A\* (utilizando as heurísticas Euclidiana e Manhattan) adaptados para ambientes tridimensionais.

Contudo, o estudo apresenta limitações de escopo, focando quase que exclusivamente em ambientes estruturados por **Voxels** (grades de blocos uniformes e cúbicos), e a execução dos testes ocorre de forma puramente sequencial e padrão. O diferencial crítico utilizando deste estudo frente a este trabalho reside em dois pontos principais:

1. **Topologia Geométrica:** Enquanto o estudo citado limita-se à rigidez ortogonal dos voxels, o gerador procedural deste estudo utiliza Ruído de Perlin e processamento de mapas de altura para extrair malhas contínuas e acidentadas. Isso eleva a complexidade do cálculo heurístico ao exigir navegação sobre relevos realistas, rampas e curvas suaves.
2. **Diversidade Algorítmica:** Estende-se a análise para algoritmos focados em otimização de espaço aberto e relaxação de linha de visão (*Any-Angle Pathfinding*), incorporando o **Theta\*** e o **Jump Point Search (JPS)**, indo muito além do escopo básico de Dijkstra e A\*.

### 5.2 Grid Maps Benchmark (2023)

O artigo *A Comparison of Pathfinding Algorithm for Code Optimization on Grid Maps*, publicado na IJACSA (Unknown; Others, 2023), realiza uma comparação estatística direta (tempo de CPU em microssegundos e contagem de nós expandidos) entre a trindade clássica de busca: A\*, JPS e Theta\*.

A principal limitação deste estudo é que os experimentos são restritos a matrizes bidimensionais planas (*2D Grid Maps*). Além disso, a execução dos algoritmos ocorre de forma síncrona e linear, avaliando buscas isoladas em cenários estáticos. Em contrapartida, os diferenciais deste estudo são:

1. **A Terceira Dimensão Físico-Espacial:** Exige a readequação volumétrica tridimensional real das heurísticas (como a distância diagonal/Chebyshev adaptada e a distância Euclidiana 3D), lidando com a complexidade geométrica do eixo Z.
2. **Arquitetura Concorrente sob Estresse:** O *benchmark* pulveriza a execução simultânea das buscas espaciais em múltiplas *threads* operárias através da biblioteca `jthread` e de filas multinível orientadas a eventos.

3. **Rigor e Segurança de Memória:** Para suportar esse ambiente de alto estresse concorrente sem mascarar os tempos de CPU com travamentos ou condições de corrida, a infraestrutura implementada utiliza ponteiros inteligentes (`shared_ptr` e `unique_ptr`), garantindo métricas puras de hardware.

5.3 Matriz de Diferenciação Técnica

A tabela a seguir consolida as lacunas da literatura e destaca as contribuições metodológicas do motor IFCG.

Tabela 1 – Matriz de diferenciação entre a proposta e trabalhos relacionados

| Critério          | Proposta (IFCG)                                        | Johansson (2024)            | Benchmark (2023)            |
|-------------------|--------------------------------------------------------|-----------------------------|-----------------------------|
| Dimensão Espacial | Malhas 3D volumétricas                                 | Voxels (blocos 3D)          | Grades 2D planas            |
| Geometria         | Contínua e procedural (Perlin)                         | Cubos fixos (discretos)     | Matrizes planas             |
| Algoritmos        | Dijkstra, A* (e modificações), JPS, Theta*, LazyTheta* | A* e Dijkstra               | A*, JPS, Theta*             |
| Arquitetura       | Multi-threaded (jthread)                               | Sequencial/Padrão           | Sequencial/Padrão           |
| Ambiente / Carga  | Renderização 3D em tempo real sob concorrência         | Execução sequencial isolada | Execução sequencial isolada |

Fonte: Elaborado pelo autor (2026).

## 6 METODOLOGIA

Esta seção detalha a metodologia adotada para a realização da pesquisa, explicando como os algoritmos de pathfinding e a infraestrutura gráfica serão implementados e testados. Este projeto foi conduzido como uma pesquisa experimental de caráter quantitativo, sendo que a implementação da infraestrutura gráfica e dos algoritmos de pathfinding ocorreu em paralelo à coleta e análise de dados. A abordagem experimental permitirá a avaliação do desempenho dos algoritmos em condições controladas, enquanto a construção do motor do zero garantirá um ambiente de teste personalizado e otimizado para as necessidades específicas deste estudo.

### 6.1 Arquitetura do Motor Gráfico (IFCG)

O motor gráfico desenvolvido para este estudo, denominado IFCG (Instituto Federal Catarinense/Computação Gráfica), foi projetado para ser uma plataforma de renderização flexível para a implementação e avaliação dos algoritmos de *pathfinding*. A arquitetura do IFCG foi cuidadosamente planejada para garantir que os testes sejam realizados em um ambiente gráfico realista, mas simples, permitindo a coleta de dados precisos sobre o desempenho dos algoritmos em cenários tridimensionais complexos.

Ele é composto por diversos módulos, cada um responsável por uma parte específica do processo de renderização e gerenciamento de recursos gráficos. A seguir, serão detalhados os principais componentes da arquitetura do motor gráfico e como eles se relacionam para criar um ambiente de teste eficiente para os algoritmos de pathfinding.

#### 6.1.1 Aplicação de conceitos

Para lidar com a complexidade inerente à renderização gráfica e ao gerenciamento de recursos, a arquitetura do IFCG foi projetada utilizando princípios de design de software e padrões de projeto. A aplicação desses conceitos é fundamental para garantir que o código seja uma simplificação das chamadas do OpenGL, mantendo a flexibilidade e a capacidade de extensão necessárias para futuras implementações e otimizações.

Como o coração do motor foi desenvolvida uma classe *Engine*, que é responsável por gerenciar o ciclo de renderização, as chamadas ao OpenGL e a interação com a janela. Essa classe encapsula toda a lógica do ciclo de renderização e fornece uma interface simples para a criação de cenários de teste, permitindo que os algoritmos de pathfinding sejam avaliados em diferentes condições e configurações.

O padrão *Singleton* foi aplicado na criação de instâncias do *Engine*, garantindo um controle centralizado sobre os recursos gráficos e a renderização. Isso evita conflitos e mantém a consistência do ambiente de renderização, permitindo que diferentes partes do sistema acessem o motor gráfico de forma segura e coordenada.

Seguindo os conceitos de Vries (2014), foi desenvolvido uma classe *Mesh*, que representa uma malha 3D e encapsula os dados necessários para renderizá-la, como

vértices, normais, coordenadas de textura e índices. Essa classe também gerencia os *buffers* VBO, EBO e VAO, garantindo que a malha seja renderizada de forma eficiente e correta.

A classe *Shader* (Vries, 2014) foi implementada para gerenciar os programas de sombreamento (*shaders*) utilizados na renderização das malhas. Ela encapsula a criação, compilação e vinculação dos *shaders*. Essa classe permite a leitura de arquivos de extensão *glsl*, que contêm o código dos *shaders*, e fornece métodos para definir uniformes e atributos de vértice, facilitando a personalização da aparência das malhas renderizadas.

### 6.1.2 Componentes do Motor Gráfico

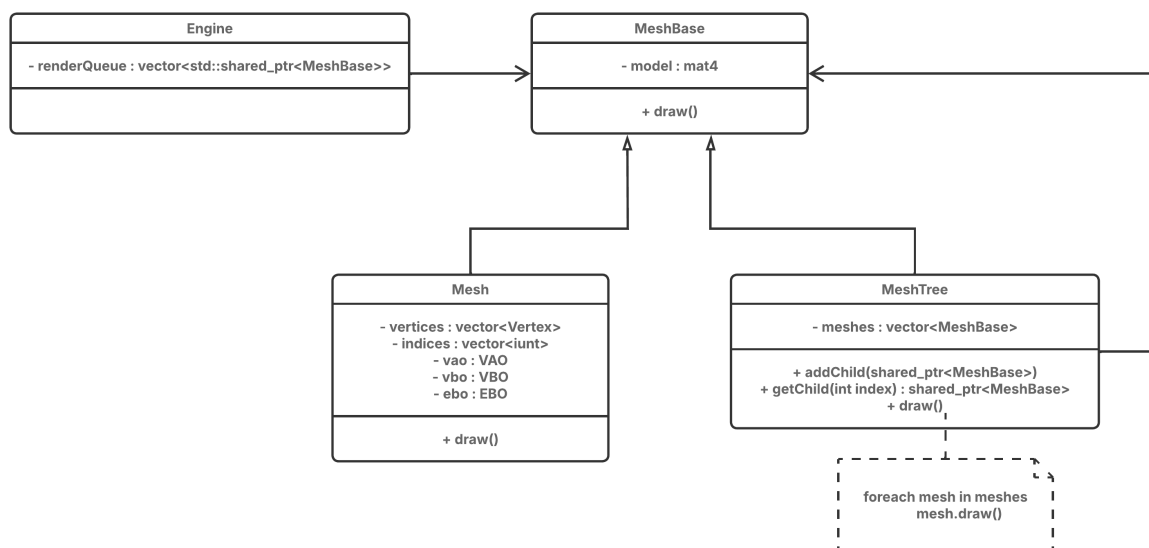
Para organizar a arquitetura do motor gráfico, foram definidos diversos módulos, cada um responsável por uma parte específica do processo de renderização e gerenciamento de recursos gráficos. A seguir, serão detalhados os principais componentes da arquitetura do motor gráfico e como eles se relacionam para criar um ambiente de teste eficiente para os algoritmos de *pathfinding*.

1. *Window*: Essa classe atua como um adaptador (*wrapper*) para a biblioteca GLFW, responsável por criar e gerenciar a janela de renderização. Ela encapsula a criação do contexto OpenGL, o gerenciamento de eventos de entrada (como teclado e mouse) e a configuração das propriedades da janela, como tamanho, título e modo de exibição.
2. *Renderer*: Essa classe é responsável por gerenciar o ciclo de renderização, incluindo a configuração do estado do OpenGL, a vinculação dos *buffers* e a emissão dos comandos de desenho. Ela também gerencia a ordem de renderização das malhas, garantindo que os objetos sejam desenhados na ordem correta e com as propriedades visuais desejadas.
3. *Input*: Essa classe é responsável por gerenciar a entrada do usuário, processando eventos de teclado e mouse. Ela fornece métodos para adicionar funções de retorno (*callbacks*) personalizadas, permitindo que a aplicação reaja a eventos de entrada de forma flexível e eficiente.

### 6.1.3 Estrutura de Renderização e Gerenciamento de Recursos

O padrão *Composite* (Gamma et al., 1994) foi utilizado para representar a hierarquia de objetos na infraestrutura gráfica, onde uma malha complexa pode ser composta por várias sub-malhas, e cada sub-malha pode ser tratada da mesma forma. Toda base de malha (*MeshBase*) contém sua própria matriz de modelo, que é responsável por armazenar as transformações de posição, rotação e escala da malha. Então as classes de malha (*Mesh*) e malha composta (*MeshTree*) herdam dessa base, permitindo que sejam tratadas de forma uniforme, independentemente de serem malhas simples ou compostas.

Figura 14 – Diagrama de classe do Composite



Fonte: Elaborado pelo autor.

Com essa arquitetura, o motor de renderização pode desenhar tanto malhas simples quanto compostas utilizando a mesma interface. Assim, permitindo criar cenários complexos com complexidade reduzida e mantendo a flexibilidade para adicionar novos tipos de malhas ou componentes gráficos no futuro.

Isso permite que o `Renderer` é capaz de percorrer a hierarquia de malhas e renderizar cada uma delas de forma eficiente, aplicando as transformações apropriadas e garantindo que a cena seja exibida corretamente na janela de renderização.

Uma fila de malhas é mantida pelo `Renderer`, permitindo que as malhas sejam adicionadas e removidas dinamicamente durante a execução do programa. Essa fila é processada a cada ciclo de renderização, garantindo que todas as malhas sejam desenhadas na ordem correta e com as propriedades visuais desejadas.

Para gerenciar a renderização foi desenvolvido um sistema de laço (*loop*) de renderização ajustado a partir de configurações pré-definidas. Isso é, foi desenvolvida uma `struct LoopConfig` que contém parâmetros como taxa de atualização desejada, limites de tempo de renderização e opções de funções `lambda` para serem executadas em momentos diferentes do ciclo de renderização. O `Renderer` utiliza essas configurações para controlar o fluxo do motor flexivelmente.

## 6.2 Implementação da Estrutura de Grafos

Foi implementada uma biblioteca de grafos em C++ para representar os caminhos possíveis dentro do ambiente tridimensional. O objetivo desta seção é detalhar a implementação da estrutura de grafos e seus algoritmos.



### 6.2.1 Representação de Grafos Direcionados e Não Direcionados

A biblioteca de grafos desenvolvida neste projeto<sup>2</sup> foi criada com duas estruturas de dados diferentes para permitir flexibilidade e desempenho: uma orientada a objetos clássica e outra focada em alto desempenho baseada em vetores contíguos na memória.

A primeira estrutura usa classes abstratas (`common::Graph`, `common::Node` e `common::Edge`), que são herdadas pelas classes de grafos direcionados (`directed`) e não direcionados (`undirected`). Nessa abordagem, cada nó e aresta é um objeto criado na memória *heap* e gerenciado com ponteiros inteligentes `std::unique_ptr` dentro de tabelas `std::unordered_map`. Embora essa organização facilite a inserção e remoção de nós, ela espalha os objetos pela memória. Isso causa frequentes falhas de cache (*cache misses*), o que diminui a velocidade dos algoritmos de busca quando o volume de dados é grande.

Para resolver esse problema de lentidão, foi criada uma segunda estrutura chamada de representação leve (*lightweight*), implementada na classe *template* `common::lwGraph<T>` e suas versões direcionada (`directed::lwGraph<T>`) e não direcionada (`undirected::lwGraph<T>`). Essa estrutura armazena os dados dos nós em um vetor contínuo na memória (`std::vector<T>`) e a lista de adjacências de forma simplificada em um vetor de vetores (`std::vector<std::vector<lwEdge>>`), onde cada aresta guarda apenas o número do nó de destino e o peso. Colocar os dados juntos de forma linear na memória melhora a localidade de cache, fazendo com que o acesso aos vizinhos de um nó seja muito mais rápido. Essa estrutura em formato de ponteiro compartilhado (`std::shared_ptr<common::lwGraph<Vertex3D>>`) também é segura para uso concorrente no gerenciador de tarefas `TaskMaster`, permitindo que várias *threads* leiam o grafo ao mesmo tempo sem precisar de travas de sincronização.

Além disso, a classe `common::lwGrid` representa uma grade simples de duas dimensões em um único vetor linear. Essa grade é usada para guardar as células livres e com obstáculos do mapa, servindo de base para algoritmos de busca em grade como o Jump Point Search (JPS).

### 6.2.2 Algoritmos de Pathfinding

Os algoritmos de busca de caminhos foram implementados utilizando a representação leve do grafo para garantir que os testes fossem executados com a maior rapidez possível.

O algoritmo Dijkstra foi implementado de forma integrada com o A\* tradicional (`util::lwAStar`). Em vez de reescrever o código do Dijkstra do zero, a função de busca chama o algoritmo A\* passando uma função heurística que sempre retorna zero ( $h(n) = 0$ ). Isso faz com que o A\* se comporte exatamente como o Dijkstra, buscando de forma uniforme em todas as direções e facilitando a manutenção do código.

---

<sup>2</sup>Disponível em: <https://github.com/andrevbastos/graph>

Além do A\* padrão, foi criada uma variação modificada chamada `util::lwAStarMod` baseada em Smolka et al. (2019). A diferença é que a estimativa da heurística é multiplicada por um fator  $M$  que muda conforme a direção do movimento. Se o movimento entre o nó atual e o seu vizinho for diagonal, o multiplicador assume o valor de  $\sqrt{2}$ , e se for ortogonal, o valor é 1.0. Essa alteração melhora a estimativa heurística nas diagonais.

O algoritmo Theta\* (`util::lwThetaStar`) foi adicionado para permitir caminhos livres de restrições de grades (Nash; Koenig, 2013). A cada passo, em vez de conectar o nó atual apenas ao vizinho imediato, o algoritmo tenta ligar o nó pai direto ao vizinho do nó atual se houver linha de visão desimpedida entre eles. A linha de visão é calculada pela função `util::lwGridLineOfSight`, que usa o algoritmo de traçado de linha de Bresenham para percorrer a grade linear e checar se todas as células no caminho são transitáveis. Para tornar a distância Euclidiana eficiente e genérica em 2D ou 3D, a função utiliza metaprogramação em C++ (`if constexpr` e `std::void_t`) para verificar na compilação se a struct do vértice possui ou não a coordenada  $z$ , calculando a distância correta sem gastar tempo de processamento em tempo de execução.

Por fim, a biblioteca traz o Jump Point Search (JPS) através da classe `util::JumpPointSearchLw`. Esse algoritmo acelera a busca pulando grandes blocos vazios na grade `common::lwGrid` até encontrar pontos de decisão (como cantos de obstáculos), evitando a inserção desnecessária de nós intermediários na fila de prioridades.

### 6.2.3 Otimizações e Estruturas de Dados Auxiliares

Além das estruturas principais e dos algoritmos de busca, foram implementados mecanismos adicionais para melhorar a qualidade dos caminhos e a velocidade de processamento.

Para resolver o problema dos caminhos em ziguezague gerados pela movimentação restrita a grades, foi criada uma rotina de suavização de caminho (`util::lwPathSmoothing`). Esse algoritmo analisa a sequência de vértices retornada pela busca e remove os nós intermediários que estão na mesma linha (colineares). A detecção de colinearidade é feita calculando o produto vetorial 2D entre os segmentos formados por três pontos consecutivos ( $P_1, P_2, P_3$ ):

$$\text{crossProduct} = (x_2 - x_1) * (y_3 - y_2) - (y_2 - y_1) * (x_3 - x_2)$$

Se o resultado for muito próximo de zero (menor que 0.001), os segmentos são considerados alinhados e o ponto intermediário  $P_2$  é descartado. Esse processo resulta em caminhos mais curtos, com menos nós e mais realistas.

Para o gerenciamento dos nós durante as buscas no A\* e no Theta\*, utiliza-se uma fila de prioridades (`std::priority_queue` da biblioteca padrão C++) configurada como um *min-heap*. A fila armazena pares contendo o custo estimado do caminho e o identificador do nó (`std::pair<double, int>`), organizando-os de forma automática para que o nó com o menor custo estimado seja sempre expandido primeiro. Isso

garante a eficiência temporal do laço principal de busca, reduzindo o custo de busca do próximo nó de  $O(V)$  para  $O(\log V)$ , onde  $V$  é o número de vértices avaliados.

### 6.3 Geração e Processamento dos Cenários

A capacidade de testar os algoritmos em uma grande variedade de mapas exige a adoção de um fluxo bem definido para a construção procedural dos terrenos, sua conversão para malhas de renderização e, mais importante, a extração de dados matemáticos para a navegação.

#### 6.3.1 Geração de Malhas 3D Complexas

Os cenários utilizados no estudo são gerados por duas abordagens complementares: a importação de mapas de altura pré-renderizados (imagens em escala de cinza, carregadas utilizando a biblioteca `stb_image`) e a geração puramente procedural baseada em Ruído de Perlin.

A geração procedural opera instanciando um mapa bidimensional  $(x, y)$  onde o algoritmo de ruído calcula uma elevação base. Esse valor é multiplicado por um fator global de intensidade, produzindo a coordenada tridimensional final do vértice  $(x, y, z)$ . Todas essas posições são agrupadas sequencialmente em *buffers* de memória para compor a malha tridimensional (Mesh) que o motor IFCG utilizará para a renderização visual do ambiente.

#### 6.3.2 Processamento de Malhas e Extração de Grafos

Para que um algoritmo de busca possa atravessar esse cenário, é necessário derivar uma representação lógica a partir dos vértices físicos. Esse processo de “extração de grafo” varre cada ponto do cenário em formato de grade e tenta conectá-lo aos seus vizinhos (adjacência horizontal, vertical e diagonal).

O diferencial nesta etapa é a limitação por inclinação. A conexão entre dois vértices (a aresta) só é considerada válida se a diferença absoluta de altura entre eles ( $|z_2 - z_1|$ ) for menor ou igual a um limite de transposição (`heightLimit`). Esse parâmetro simula a capacidade física de uma entidade escalar um obstáculo, tornando encostas muito íngremes e penhascos intransponíveis. Uma vez validada a conexão, o custo dessa aresta é assinalado como a distância Euclidiana 3D real entre os dois pontos, refletindo o peso natural da elevação.

#### 6.3.3 Configuração de Cenários de Teste e Parâmetros

O desenho experimental prevê que os testes sejam automatizados em sequências de estresse parametrizado, definindo três eixos de configuração centrais:

- **Escala:** Varia progressivamente o tamanho (largura e profundidade) da malha e, por consequência, o número absoluto de nós a serem expandidos pelos algoritmos.
- **Lacunaridade:** Altera a frequência do Ruído de Perlin. Uma frequência mais alta cria terrenos mais esburacados, com “ruídos” intensos e obstáculos constantes, o

que eleva a dificuldade heurística e bloqueia linhas de visão diretas (desafiando algoritmos como o Theta\* e o JPS).

- **Persistência:** Manipula a amplitude das funções de ruído, modificando a “suavidade” do terreno. Variações bruscas na persistência intensificam as elevações, forçando interrupções constantes pela trava física do `heightLimit`.

Tais parâmetros são injetados diretamente na configuração de geração procedural a cada iteração de teste, assegurando amostragem abrangente para análises estatísticas.

## 6.4 Implementação do Sistema Concorrente

Durante a execução de simulações em tempo real, o motor gráfico IFCG precisa manter a taxa de quadros por segundo (FPS) estável. Para isso, foi implementado um sistema de concorrência que permite que tarefas como geração de ruído, processamento de mapas de altura e extração do grafo de adjacência sejam realizadas em *threads* separadas, sem interferir na renderização principal.

### 6.4.1 Adaptação para Ambientes de Renderização

Para viabilizar a execução de simulações interativas sem prejuízo à taxa de quadros e à fluidez visual, o sistema de concorrência foi adaptado para contornar as limitações de acesso do OpenGL. O motor gráfico IFCG opera sob estricção de contexto gráfico de *thread* única, no qual o contexto de renderização é vinculado a uma única *thread* ativa (*thread* principal). Qualquer tentativa de invocar funções do OpenGL ou manipular estruturas de dados de GPU (tais como *VAOs* e *VBOs*) a partir de *threads* secundárias resulta em comportamento indefinido ou no encerramento abrupto do programa por violações de acesso à memória (erros SIGSEGV).

A adaptação implementada realiza um desacoplamento completo entre o fluxo de renderização e quaisquer tarefas secundárias. A geração de ruído, o processamento de mapas de altura e a extração do grafo de adjacência são executados de maneira puramente assíncrona. Essas tarefas não possuem dependências ou chamadas diretas ao OpenGL. Uma vez que o processamento do terreno e a extração do grafo são concluídos, os dados são sincronizados com a *thread* principal. Esta se encarrega de efetuar as chamadas de alocação e atualização de *buffers* na GPU dentro do ciclo síncrono de renderização, garantindo a integridade do contexto OpenGL e mantendo estável a taxa de quadros por segundo da visualização tridimensional.

### 6.4.2 Implementação de Multithreading e Testes de Estresse

A orquestração das tarefas em segundo plano é gerenciada pela classe `TaskMaster`, desenvolvida como um escalonador de tarefas concorrente baseado em filas de prioridade multinível (High, Medium e Low) e um *thread pool* dinâmico. O construtor do `TaskMaster` consulta a capacidade física do processador por meio de `std::thread::hardware_concurrency()`, instanciando  $N - 1$  *threads* operárias (onde  $N$

é o total de núcleos lógicos disponíveis) para resguardar a capacidade de processamento da *thread* principal e evitar travamentos na interface gráfica do usuário. As *threads* operárias são implementadas como objetos `std::jthread` (introduzidos no padrão C++20), que utilizam o comportamento RAI para garantir que sejam finalizadas corretamente na destruição do escalonador.

**Figura 15 – Diagrama de classes da arquitetura do TaskMaster**



O método `addTask` é o ponto de entrada para a submissão de tarefas concorrentes. Utilizando modelos de programação (*templates*) e metaprogramação em tempo de compilação (`if constexpr`), juntamente com o traço de tipo `std::is_invocable_v`, o método diferencia funções que aceitam um `std::stop_token` daquelas que não requerem controle de cancelamento, envolvendo estas últimas em um adaptador (*wrapper*). Isso confere flexibilidade ao sistema, permitindo que tarefas de longa duração verifiquem periodicamente se uma interrupção foi solicitada, enquanto tarefas curtas e simples podem ser executadas sem essa complexidade adicional.

Cada *thread* operária executa um laço contínuo que aguarda por novas tarefas utilizando uma `std::condition_variable_any`. A lógica de seleção de tarefas prioriza sempre as filas de maior importância: o trabalhador verifica sequencialmente as filas 0 (High), 1 (Medium) e 2 (Low), extraindo a primeira tarefa disponível na fila de maior prioridade encontrada. Mesmo que um *lock* tenha sido adquirido por meio de `std::unique_lock<std::mutex>`, a função `wait` da variável condicional libera o *lock* enquanto a *thread* está bloqueada, permitindo que a *main thread* adicione novas tarefas concorrentes sem contenção desnecessária. Quando a *thread* operária é acordada, o *lock* é automaticamente re-adquirido para a extração segura da tarefa.

Para monitoramento e testes de estresse do *thread pool*, o **TaskMaster** incorpora a função de escrita `drawWorkers()`. Protegida por um semáforo de exclusão mútua (`printMtx`), ela atualiza e imprime no console o estado de cada *thread* trabalhadora (usando os caracteres H, M, L e - para representar atividades de alta, média, baixa prioridade ou ociosidade, respectivamente), oferecendo *feedback* visual instantâneo e contínuo durante a execução em lote das simulações.

### 6.4.3 Controle de Recursos do Sistema Operacional

Para garantir que a execução paralela não comprometa a estabilidade do sistema ou a fluidez da interface gráfica, a arquitetura adota estratégias rigorosas de controle de recursos em nível de software e hardware. Ao limitar o *pool* de trabalhadores ao total de núcleos físicos menos um ( $N - 1$ ), reduz-se o custo de trocas de contexto (*context switching*) e a disputa por cache L3 entre as *threads* operárias e o motor de renderização. Além disso, o suporte ao cancelamento cooperativo via `std::stop_token` permite interromper de forma imediata tarefas obsoletas, evitando o desperdício de ciclos de CPU em processamentos desnecessários.

Adicionalmente, a fim de obter medições estatísticas limpas durante as baterias de testes estatísticos, a aplicação realiza o controle de afinidade de CPU por meio de chamadas de baixo nível do sistema operacional (utilizando `pthread_setaffinity_np` envelopado na função `pinThreadToCore`). Enquanto as tarefas de geração do mapa de ruído e construção do grafo ocorrem concorrentemente sob o gerenciamento do `TaskMaster` em múltiplos núcleos, a *main thread* sincroniza a conclusão do lote de processamento e vincula sua execução estritamente a um núcleo de processador isolado (como o núcleo 2) para executar os algoritmos de busca ( $A^*$ , Dijkstra e JPS). Esse isolamento de afinidade evita flutuações e ruído causados pelo agendador do sistema operacional, blindando os *benchmarks* estatísticos contra perturbações dinâmicas de concorrência.

## 6.5 Desenho Experimental e Coleta de Dados

A coleta de dados foi projetada para avaliar o impacto computacional dos algoritmos em terrenos de complexidade variável.

### 6.5.1 Ambiente de Teste

A avaliação empírica do sistema foi executada em um computador pessoal com processador Intel Core i5-1235U de 12ª geração. Esse processador possui uma arquitetura híbrida contendo 10 núcleos físicos (2 núcleos de alta performance e 8 núcleos de alta eficiência) que totalizam 12 *threads* lógicas de execução, operando com frequência de clock máxima de 4,40 GHz e 12 MB de memória cache. O sistema possui 16 GB de memória RAM DDR4 operando a 3200 MHz em canal duplo (dual-channel). A renderização das malhas 3D e a rasterização do motor gráfico foram processadas por uma GPU integrada Intel Iris Xe Graphics rodando a uma frequência dinâmica máxima de 1,20 GHz.

Em termos de software, os *benchmarks* foram realizados sob o sistema operacional Arch Linux rodando o *Kernel* estável 6.15.9. Para reduzir perturbações externas nos testes de performance, o agendador de energia do processador foi configurado manualmente para o modo de desempenho máximo (*performance*), mantendo os clocks elevados e estáveis. O código-fonte foi desenvolvido no padrão C++20 (`-std=c++20`) e compilado com o GNU Compiler Collection (GCC) versão 14.1.

Para extrair o máximo desempenho do hardware nos testes comparativos, o binário final foi construído utilizando a *flag* de otimização agressiva de velocidade de execução `-O3` no perfil de lançamento (`Release`), além de parâmetros estritos de compilação que garantem a segurança do código (`-Wall`, `-Wextra`, `-Wpedantic`, `-Werror`). A renderização gráfica foi programada sobre a especificação do OpenGL versão 4.6 (*Core Profile*), usando a biblioteca GLFW 3.4 para o controle de janelas e contexto gráfico, e a biblioteca GLM 1.0.3 para o processamento algébrico de matrizes e vetores tridimensionais.

### 6.5.2 Configuração dos Testes e Métricas Coletadas

Para analisar o desempenho, foram selecionadas métricas que avaliam tanto a eficiência computacional quanto a qualidade da solução e a estabilidade do sistema:

- **Tempo de Execução:** Mede o intervalo gasto na geração de mapas, construção de grafos e busca de caminhos.
- **Responsividade (FPS):** A taxa de quadros por segundo é monitorada para validar a eficácia do isolamento da *main thread*.
- **Consumo de Memória:** Avalia o impacto das estruturas de dados (malhas e grafos) no uso de RAM.
- **Eficiência do Cache:** Analisa a taxa de acertos e falhas na CPU, fornecendo *insights* sobre a localidade de referência dos algoritmos.
- **Métricas de Busca:** Incluem o número de nós expandidos e o custo total do caminho para validar a otimalidade.

Com essas métricas, será possível realizar uma análise abrangente do desempenho dos algoritmos de pathfinding em diferentes cenários, identificando as condições sob as quais cada algoritmo se destaca ou apresenta limitações. Como resultado, espera-se fornecer recomendações práticas para a escolha de algoritmos em aplicações de navegação tridimensional em tempo real para diferentes circunstâncias.

### 6.5.3 Execução dos Testes e Registro dos Resultados

O experimento utiliza uma bateria de testes organizada em três eixos principais de variação: escala, lacunaridade e persistência. A aquisição dos dados estatísticos é realizada de forma paralelizada para otimizar o tempo total de experimentação e garantir que o motor gráfico permaneça responsivo.

O fluxo de execução é gerenciado pelo TaskMaster, que agrupa as tarefas de cada repetição do experimento:

1. **Geração do Mapa (Alta Prioridade):** O mapa de ruído de Perlin é gerado na fila `Priority::High`.
2. **Construção do Grafo e Malha (Média Prioridade):** A extração da estrutura de dados para busca ocorre na fila `Priority::Medium`.
3. **Exportação de Dados (Baixa Prioridade):** O salvamento das representações visuais (PNG) é feito na fila `Priority::Low` por ser uma operação de I/O lenta.

A *main thread* utiliza um mecanismo de sincronização baseado em `std::condition_variable` para aguardar a conclusão de um lote completo de processamento (passo do experimento). Somente após todos os grafos estarem prontos e alocados, a *thread* principal executa os algoritmos de busca ( $A^*$ , Dijkstra, etc.) e registra os dados estatísticos, garantindo que a medição de performance não sofra ruído devido à contenção de recursos do processamento paralelo.

Para garantir um ambiente de teste com o mínimo de interferências externas, certas configurações do sistema são ajustadas. Um shell script é utilizado para configurar a afinidade de CPU do processo, limitando-o a um subconjunto específico de núcleos para evitar interferências de outros processos do sistema operacional. Além disso, o sistema é configurado para minimizar a interferência de serviços em segundo plano, garantindo que os dados coletados reflitam o desempenho real dos algoritmos sob as condições controladas do experimento.



## 7 RESULTADOS

Foi possível cumprir com todas as etapas definidas no cronograma de atividades até o momento. A base principal e mais pesada da pesquisa já está consolidada e funcionando de maneira estável.

Entre as metas alcançadas, destaca-se o desenvolvimento completo do motor gráfico (IFCG) e de toda a infraestrutura para testes de algoritmos de busca. Isso inclui a criação do sistema de tarefas concorrentes (TaskMaster) e da representação otimizada de grafos. Além disso, os algoritmos fundamentais de busca de caminhos (Dijkstra, A\*, Jump Point Search e Theta\*) foram totalmente implementados e testados, contando também com a rotina de suavização de caminhos.

Os testes iniciais mostraram que a estrutura desenvolvida funciona conforme esperado. O sistema consegue simular cenários acidentados em tempo real através do OpenGL e executar múltiplos benchmarks em paralelo sem travar a interface visual e sem corromper a memória. Isso prova que as decisões de arquitetura de software tomadas no início do projeto foram corretas, gerando uma base sólida para as próximas etapas.

Futuramente, os esforços serão direcionados para o desenvolvimento de técnicas avançadas de representação espacial e a exploração de novos algoritmos. O objetivo será expandir o sistema para suportar:

- **NavMeshes** (Malhas de Navegação): permitindo a movimentação livre e contínua de entidades sobre superfícies 3D complexas.
- **Octrees** (Árvores Octais): para o particionamento espacial tridimensional eficiente de ambientes de grande escala.
- **Voxels** (Mapeamento Volumétrico): para representar terrenos volumétricos de forma flexível.

Por fim, pretende-se pesquisar e integrar novos algoritmos de busca de caminhos para ampliar o comparativo estatístico nas novas estruturas espaciais.

## 8 CRONOGRAMA

**Tabela 2 – Cronograma de execução das atividades**

| <b>ETAPAS</b>                                                         | <b>PERÍODOS</b>     |
|-----------------------------------------------------------------------|---------------------|
| Revisão de Bibliografia                                               | Jan/2026 a Fev/2026 |
| Arquitetura Base (IFCG) e Estrutura de Grafos                         | Fev/2026 a Abr/2026 |
| Heurísticas e Implementação Base (A*, Dijkstra, JPS, Theta*)          | Mar/2026 a Mai/2026 |
| Geração de Cenários de Teste e Malhas 3D                              | Abr/2026 a Mai/2026 |
| Multithreading e Testes de Estresse                                   | Mai/2026 a Jun/2026 |
| Planejamento da Coleta de Dados e Análise Estatística                 | Mai/2026 a Jun/2026 |
| Redação e Revisão do Pré-Projeto                                      | Abr/2026 a Jun/2026 |
| Defesa do Pré-Projeto                                                 | Jun/2026            |
| Implementação de NavMeshes e Otimização 3D                            | Jul/2026 a Ago/2026 |
| Otimização de Concorrência e Testes de Estresse                       | Ago/2026            |
| Implementações Específicas (D* Lite, Busca Bidirecional, Lazy Theta*) | Ago/2026 a Set/2026 |
| Coleta Automatizada de Métricas e Dados                               | Set/2026 a Out/2026 |
| Análise Estatística e Visual dos Resultados                           | Out/2026 a Nov/2026 |
| Redação da Monografia Final                                           | Out/2026 a Nov/2026 |
| Revisão e Entrega Final                                               | Nov/2026            |
| Defesa do TCC                                                         | Dez/2026            |

Fonte: Elaborado pelo autor (2026).

## REFERÊNCIAS

DOMINGUES, Jurandir Junior. **Material didático**. Blumenau, SC, 2026.

GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S.l.]: Addison-Wesley, 1994.

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

GOOGLE LLC. **GoogleTest User's Guide**. Disponível em: <https://google.github.io/googletest/>. Acesso em: 23 jun. 2026.

HART, Peter E. ; NILSSON, Nils J. ; RAPHAEL, Bertram. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100–107, 1968.

THE IMPERIAL LIBRARY. **Maps of Solstheim**. Disponível em: <https://www.imperial-library.info/content/maps-solstheim>.

JOHANSSON, E.; OTHERS. **Adapting Pathfinding Algorithms for 3D Environments: Performance Analysis and Real-World Applications**. Master's thesis—[S.l.: S.n.].

LADEIRA, Ricardo de la Rocha. **Material didático**. Blumenau, SC, 2026.

MADUSHANKA, Tiroshan; MADUSHANKA, Sakuna. **Multi-threaded Recast-Based A\* Pathfinding for Scalable Navigation in Dynamic Game Environments**. Disponível em: <https://arxiv.org/abs/2602.04130>.

NASH, Alex; KOENIG, Sven. Any-Angle Path Planning. **AI Magazine**, v. 34, n. 4, p. 85–107, 2013.

NOBES, Thomas K. *et al.* The JPS Pathfinding System in 3D. **Proceedings of the International Symposium on Combinatorial Search**, v. 15, n. 1, p. 145–152, jul. 2022.

PARDEDE, Sara Lutami *et al.* A review of pathfinding in game development. **Journal of Computer Engineering**, v. 1, n. 01, p. 47–56, 2022.

PATEL, Amit J. **Making maps with noise functions**. Disponível em: <https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution>. Acesso em: 7 maio. 2026.

PERLIN, Ken. An Image Synthesizer. *In*: SIGGRAPH '85. New York, NY, USA: Association for Computing Machinery, 1985. Disponível em: <https://doi.org/10.1145/325334.325247>

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: uma abordagem moderna**. 3. ed. Rio de Janeiro: Elsevier, 2013.

SMOŁKA, Jakub *et al.* A\* pathfinding algorithm modification for a 3D engine. **MATEC Web Conf.**, v. 252, p. 3007, 2019.

UNKNOWN; OTHERS. A Comparison of Pathfinding Algorithm for Code Optimization on Grid Maps. **International Journal of Advanced Computer Science and Applications**, v. 14, n. 1, 2023.

VRIES, Joey de. **Learn OpenGL: Enjoy learning modern OpenGL**. Disponível em: <https://learnopengl.com/>. Acesso em: 18 abr. 2026.

WILLIAMS, Anthony. **C++ Concurrency in Action**. 2nd. ed. [S.l.]: Manning Publications, 2019.

ZIPPED. **C++: Perlin Noise Tutorial**. Disponível em: <https://www.youtube.com/watch?v=kClAHqb60Cw>. Acesso em: 7 maio. 2026.